EASTERN INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

DEPARTMENT OF COMPUTER NETWORKS AND DATA COMMUNICATION

PROJECT REPORT

AN EXPLORATORY STUDY OF MCP SERVER SECURITY: VULNERABILITY ANALYSIS AND DETECTION EVALUATION

Students

Pham Nguyen Khanh – 2331220008

Lu Hoang Duy – 2331220031

Truong Thi Van – 2331220034

Supervisor

Dr. Vu Duc Ly

Ho Chi Minh, June 2026

ABSTRACT

The Model Context Protocol (MCP) has rapidly become the dominant standard for connecting Large Language Model (LLM) agents to external tools and data sources, but its growth has outpaced the security infrastructure surrounding it. Unlike mature software ecosystems with centralized review pipelines, MCP marketplaces currently accept third-party servers with minimal auditing, creating an attack surface that conventional application security tooling was never designed to address. This report presents an exploratory study of MCP server security, combining an architectural analysis of the protocol with hands-on detection experiments. We first examine the structural components of an MCP server, the formal threat model governing malicious server behavior, and the real-world deployment mechanisms through which servers reach users. We then analyze three production-grade MCP servers as architectural baselines, alongside three confirmed real-world malicious servers (Postmark-MCP, mcp-pdftool-plus, and mcp-server-todo) and three academic Proof-of-Concept datasets (damn-vulnerable-MCP-server, the Song et al. attack suite, and MCPSecBench), mapping each onto a component-centric attack taxonomy.

Building on this foundation, we evaluate two representative detection tools, Snyk (a general-purpose SAST/SCA platform) and Cisco MCP Scanner (a YARA-based, MCP-specific scanner), against all three PoC datasets. Our results show a consistent pattern: both tools reliably detect conventional code-level vulnerabilities, including command injection, path traversal, SSRF, and hardcoded secrets, when such flaws exist as literal patterns in source code, but detection collapses entirely against attacks whose payload consists of natural-language instructions embedded in tool descriptions or runtime responses. Cisco MCP Scanner's keyword-based detection rate falls from 16% on a dataset with deliberately obvious naming to 0% against two independently constructed datasets using realistic, professional-sounding tool names. Snyk, despite finding 71 code-level issues in one dataset, returns zero findings against the three files that constitute that dataset's actual semantic attack contribution. We attribute this gap to a structural limitation shared by both tools: neither observes a server's behavior at execution time, which leaves an entire class of conditionally-triggered and instruction-based attacks outside what static, pre-execution scanning can detect. We conclude with a discussion of mitigation directions, including behavioral deviation analysis and hybrid defensive architectures, as a path toward closing this detection gap.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, Dr. Vu Duc Ly, for his guidance, patience, and valuable feedback throughout the course of this project. His insights were instrumental in shaping the direction and scope of our research.

We also thank the School of Computing and Information Technology and the Department of Computer Networks and Data Communications at Eastern International University for providing the academic environment and resources that made this study possible.

TABLE OF CONTENTS

LIST OF FIGURES AND TABLES

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

No.	Term	Meaning
1	**A1–A12**	Component-centric attack taxonomy categories (Section 2.1.4)
2	**API**	Application Programming Interface
3	**ASR**	Attack Success Rate
4	**BCC**	Blind Carbon Copy
5	**C2**	Command and Control
6	**CG**	Call Graph
7	**CI/CD**	Continuous Integration / Continuous Deployment
8	**CLI**	Command-Line Interface
9	**COM**	Component Object Model
10	**CORS**	Cross-Origin Resource Sharing
11	**CPG**	Code Property Graph
12	**CVE**	Common Vulnerabilities and Exposures
13	**DNS**	Domain Name System
14	**DoS**	Denial of Service
15	**DOM**	Document Object Model
16	**EDR**	Endpoint Detection and Response
17	**ETH**	Ether (Ethereum's native cryptocurrency)
18	**F1**	F1-score (harmonic mean of precision and recall)
19	**HTTP**	Hypertext Transfer Protocol
20	**IP**	Internet Protocol
21	**IRB**	Institutional Review Board
22	**JSON**	JavaScript Object Notation
23	**JSON-RPC**	JSON Remote Procedure Call
24	**LLM**	Large Language Model
25	**MCP**	Model Context Protocol

No.	Term	Meaning
26	**MD5**	Message Digest Algorithm 5
27	**MER**	Malicious External Resources
28	**npm**	Node Package Manager
29	**OAuth**	Open Authorization
30	**OS**	Operating System
31	**OSV**	Open Source Vulnerabilities (database)
32	**PA**	Puppet Attack
33	**PDF**	Portable Document Format
34	**PoC**	Proof-of-Concept
35	**PyPI**	Python Package Index
36	**RCE**	Remote Code Execution
37	**RQ**	Research Question
38	**RR**	Refusal Rate
39	**SAST**	Static Application Security Testing
40	**SCA**	Software Composition Analysis
41	**SDK**	Software Development Kit
42	**SIGINT**	Signal Interrupt
43	**SSE**	Server-Sent Events
44	**SSH**	Secure Shell
45	**SSRF**	Server-Side Request Forgery
46	**stdio**	Standard Input/Output
47	**TLS**	Transport Layer Security
48	**TPA**	Tool Poisoning Attack
49	**UI**	User Interface
50	**URI**	Uniform Resource Identifier
51	**URL**	Uniform Resource Locator
52	**USD**	United States Dollar
53	**VM**	Virtual Machine

No.	Term	Meaning
54	**WASM**	WebAssembly
55	**YARA**	Yet Another Recursive Acronym

CHAPTER 1. INTRODUCTION

1.1. Problem Statement

The rapid advancement of Large Language Models (LLMs) has transformed them from simple conversational interfaces into autonomous AI agents capable of performing complex, real-world tasks. To achieve this, agents require seamless access to external data sources and specialized tools, leading to the development of various integration frameworks. Among these, the Model Context Protocol (MCP) has emerged as a dominant open standard, providing a "plug-and-play" architecture that allows LLMs to dynamically interact with diverse systems [1].

MCP adoption has significantly outpaced the development of its security infrastructure. Mature software ecosystems often rely on centralized app stores and review processes; the MCP ecosystem still lacks comparable standardized review mechanisms [1]. Most users obtain MCP servers through public registries or automated package managers built for discovery and convenience, with no behavioral audit before code reaches a user's machine.

The gap matters because MCP servers often receive broad, repeated authority through an autonomous LLM acting on a user's behalf. Their access can include local files, credentials, browser sessions, financial APIs, or internal corporate systems, giving them a wider operational role than many ordinary npm or PyPI dependencies. Real incidents show the practical impact. A typosquatted email connector exfiltrated outbound correspondence for weeks through a single added line hidden inside otherwise legitimate functionality. A todo-list server with two ordinary-looking tools created a path toward cryptocurrency-wallet theft on machines that installed it. These cases did not depend on advanced exploitation. They depended on trusted tools remaining ordinary until a specific action exposed the hidden behavior.

This exposes organizations adopting MCP to a category of supply chain risk that existing security practices were not built to address. Enterprises already run static analysis, dependency scanning, and signature-based review as standard practice across their software supply chain, and a natural assumption is that the same tooling extends to cover MCP servers, since these are, after all, third-party code being pulled into a production environment. The experiments in this report test that assumption directly: first by identifying the MCP attack surfaces that scanners need to reason about, then by measuring what Snyk and Cisco MCP Scanner detect, and finally by explaining where their results collapse against semantic, instruction-based attacks.

1.2. Research Questions

To align the report with the empirical findings in Chapter 5, this study is guided by three Research Questions (RQs):

- **RQ1:** Which MCP components and deployment mechanisms create the main attack surfaces that malicious servers can exploit?
- **RQ2:** How effectively do Snyk and Cisco MCP Scanner detect vulnerabilities that are visible as source-code flaws, vulnerable dependencies, or explicit metadata/signature patterns?
- **RQ3:** Why do these scanners fail against semantic and context-dependent MCP attacks, and what defensive directions are needed to reduce that gap?

1.3. Contributions

This report makes the following key contributions to the field of AI agent security:

1. **Systematization of MCP Vulnerabilities:** We provide a comprehensive component-centric analysis and threat model of malicious MCP servers, bridging theoretical attack vectors with real-world deployment mechanisms.
2. **Empirical Assessment of Existing Scanners:** We conduct a hands-on evaluation of CI/CD-ready security tools (Snyk SAST/SCA and Cisco MCP Scanner) against both traditional vulnerabilities and academic Proof-of-Concept (PoC) semantic attack datasets.
3. **Identification of the Semantic Detection Gap:** We supply empirical evidence demonstrating the structural inability of static and metadata scanners to detect conditionally triggered, natural-language instructions (semantic attacks), and outline actionable hybrid and behavioral defensive architectures to mitigate this gap.

1.4. Project Objectives

This project pursues four objectives, structured to move from foundational understanding toward empirical evaluation.

1. **Investigate a conceptual foundation for MCP security.** Examine the protocol's core architecture, its six structural components, the formal threat model governing adversarial MCP servers, and a component-centric taxonomy of attack categories, providing the analytical framework used throughout the rest of the report.
2. **Characterize malicious MCP servers across both real-world and academic contexts.** Analyze production-grade MCP servers as architectural baselines, document

confirmed real-world malicious servers that have affected actual users, and map three academic Proof-of-Concept datasets onto the attack taxonomy established in the first objective.

3. **Survey existing MCP detection methodologies and select representative tools for empirical testing.** Review the design and known limitations of current MCP security scanners, then evaluate two tools with fundamentally different detection philosophies, Snyk (general-purpose static analysis) and Cisco MCP Scanner (MCP-specific, signature-based scanning), against the three PoC datasets characterized above.

4. **Identify and explain detection gaps, then propose mitigation directions.** Synthesize the empirical results into a clear account of what each tool reliably catches and what it structurally cannot, and outline defensive approaches, including behavioral deviation analysis, that could close the gap this report identifies.

1.5. Project Scope

This project is scoped around three malicious MCP server datasets and two detection tools, evaluated together to produce a comparative picture of current scanning capability rather than an exhaustive benchmark of every available tool.

In scope:

- Three PoC datasets spanning a deliberate range of attacker sophistication: `damn-vulnerable-MCP-server` [7], an educational benchmark with intentionally labeled vulnerabilities; the Song et al. attack suite [3], built around realistic, professionally worded tool descriptions; and MCPSecBench [8], which extends testing into CVE-based remote code execution and network-layer exploits.

- Empirical scanning of all three datasets using Snyk (both SCA dependency scanning and SAST source code analysis) and Cisco MCP Scanner, restricted to its YARA and vulnerable-package analyzers.

- A literature-based survey, without hands-on testing, of three additional detection approaches, MCP-Scan, AI-Infra-Guard, and Connor, used to contextualize the two tools tested empirically and to identify what a more capable detection architecture would need to address.

- Documentation of confirmed real-world malicious MCP servers (Postmark-MCP, mcp-pdftool-plus, mcp-server-todo) as supporting evidence that the attack patterns examined in this report have caused actual harm, drawn from public disclosures and prior research rather than independently reproduced by this project.

Out of scope:

- Designing, implementing, or training a new MCP detection tool. Connor's behavioral deviation architecture is discussed as a point of comparison, but this project does not attempt to build an equivalent system.
- Empirical testing of Cisco MCP Scanner's LLM Analyzer or Cisco AI Defense API engines, which require API access beyond this project's available infrastructure; all Cisco results in this report reflect signature-based detection only.
- Re-scanning the network-layer attacks within MCPSecBench (man-in-the-middle interception, DNS rebinding) as standard MCP-layer findings, since these operate beneath the component structure either tool is designed to inspect.
- Independent reproduction or exploitation testing of the three real-world malicious servers; all technical detail on these incidents is drawn from existing public research and security disclosures.

1.6. Report Structure

The remainder of this report is organized as follows.
Chapter 2 (Background) establishes the foundational knowledge needed to understand the rest of this report: the MCP architecture, threat model, attack taxonomy, real-world deployment mechanisms, confirmed malicious MCP servers observed in practice, and academic proof-of-concept benchmarks that formalize specific attack mechanisms.
Chapter 3 (Related Works) reviews the existing MCP security scanning literature and the empirical attack-evaluation study most directly comparable to this report's own contribution, explicitly positioning this study against each.
Chapter 4 (Proposed Approach, Analysis, Design, Implementation) describes this project's proposed approach and experimental setup, including production-grade architectural baselines, target datasets, and selected tools.
Chapter 5 (Results and Discussion) answers the three research questions in sequence: the MCP attack surfaces observed, the detection capability of the two scanners, and the semantic detection gap that remains.
Chapter 6 (Conclusion and Future Works) summarizes the findings, proposes mitigation strategies, outlines limitations, and suggests directions for future research.

CHAPTER 2. BACKGROUND

This chapter establishes the background knowledge needed to follow the rest of this report: what an MCP server is and how it is structured, the threat model this report adopts, the taxonomy used to classify adversarial behavior, the mechanisms through which servers actually reach users, and concrete evidence that the resulting risks are not hypothetical. It covers the protocol's architecture and attack taxonomy, real-world deployment mechanisms, malicious servers observed in the wild, and academic proof-of-concept benchmarks that formalize specific attack mechanisms. The next part of the report then positions this background against existing scanners and empirical attack studies.

2.1. MCP Background and Threat Landscape

2.1.1. MCP Definition and Core Architectural Framework

The Model Context Protocol (MCP) is an open standard built to solve a specific integration problem: LLMs and the external tools or data sources they need to act on the world had no common interface [1, 2]. Before MCP, connecting an agent to a new capability meant writing a custom, ad-hoc API integration for each one, work that grew development overhead with every new connection and made the resulting software harder to reason about from a security standpoint. MCP replaces this with a single, modular client-server pattern, structured around three entities that together manage an agent session's runtime lifecycle.

The MCP Host is the application the LLM actually runs inside, Claude Desktop or Cursor being the two most common examples. It orchestrates the session as a whole: holding conversational state, enforcing the system prompt, evaluating what a tool call returns, rendering resources, and drawing the security boundary meant to keep untrusted server data from contaminating the user's own environment [1, 2].

The MCP Client sits inside the host and does the actual talking to a server. It maintains a persistent, bi-directional connection, translates the model's abstract intent into structured protocol messages, manages the transport layer (stdio for local servers, Server-Sent Events for remote ones), and runs the capability-discovery handshake that tells the host what a server can do [1, 2].

The MCP Server is the decoupled piece doing the real work: a standalone service exposing a specific domain, a curated set of tools, resources, or data access logic. It advertises what it offers, executes whatever the client asks of it, and returns the result as structured context the host folds back into the model's reasoning [1, 2].

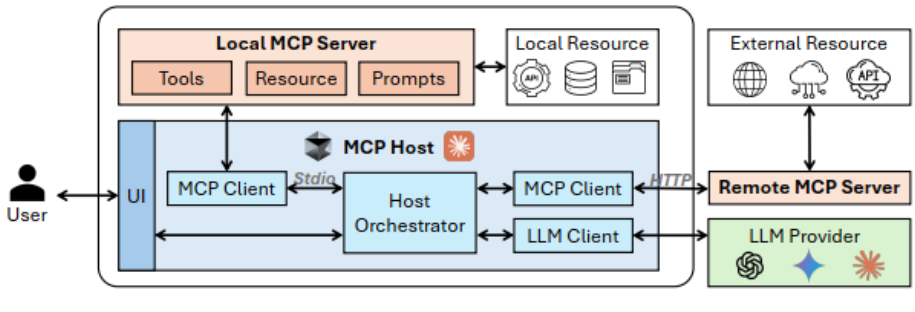

Figure 1: Core architectural framework of the Model Context Protocol, highlighting the interaction boundaries between Host, Client, and Server systems. Redrawn from Zhao et al. [1].

2.1.2. Granular Component Structure of MCP Servers

An MCP server is not a single black box; it is built from six components, and each one represents a distinct surface a malicious developer could target [1, 2].

Metadata is everything about the server meant for human or system consumption: README documentation, promotional text, and the structured JSON fields (name, version, authorization settings) used during capability discovery [1]. Configuration governs how the server actually launches: executable paths, command-line arguments, environment variables, and transport settings such as ports or remote URLs [1, 2]. Initialization logic is the bootstrap code that runs first, the handshake that negotiates protocol version and capability profiles with the client before any tool is ever exposed to the agent [1].

Tools are the invokable functions a server exposes for the LLM to call. Each one carries a name, a human-readable description, and a JSON schema defining the parameters the model must supply before invocation [1, 2]. Resources are read-only data objects addressed by URI, letting a server push static or dynamic content directly into the model's context without triggering any state change [1, 2]. Prompts are reusable templates a server can register to steer how a user phrases a request for a given task [1, 2].

2.1.3. Formal Threat Modeling and Attacker Dimensions

A useful security analysis needs a clearly bounded threat model rather than a vague sense that "servers can be malicious" [1, 2]. The threat actor assumed throughout this report is a third-party developer distributing MCP servers through open ecosystems, public repositories, or unvetted marketplaces, someone with working knowledge of LLM tool-calling conventions and prompt engineering [1].

This attacker has full control over the server's source code, metadata, and promotional descriptions while remaining within valid protocol syntax. The server must connect cleanly and appear structurally compliant, since a failed handshake would prevent the attack from reaching the user. The objectives behind that control are local system compromise, data exfiltration, and steering the model's behavior past its own alignment boundaries [1].

Everything else in this model is assumed trustworthy. The host application, the LLM's training weights, and the user's own instructions are taken as uncompromised; the malicious server is the one active, stateful threat sitting inside an otherwise trusted perimeter [1].

2.1.4. Comprehensive Taxonomy of Attack Categories

Mapping where an attacker can actually act within this model produces a component-centric taxonomy of twelve categories, A1 through A12, spread across the initialization, interaction, and output phases of a server's lifecycle [1].

Setup and Instantiation Attacks

The earliest opportunities sit at installation. A1, Server Metadata Poisoning, covers an adversary crafting a deceptive server name or description to mimic a trusted utility, fooling either a host's configuration scanner or the human installing it [1]. A2, Server Configuration Abuse, exploits the implicit trust the host extends to a server's launch parameters: shell commands that mount the host's root directory, or environment manipulation that establishes a persistent background process [1]. A3, Initialization Logic Attack, embeds malicious payloads directly in the server's bootstrap routine, so arbitrary code runs the moment the transport connects, before any user session or tool invocation has even begun [1].

Tool-Driven Manipulation Vectors

The second wave targets tools during live use. A4, Tool Metadata Poisoning, embeds persuasive or manipulative language inside a tool's registration block, distorting the LLM's attention enough to over-collect sensitive data or favor a malicious tool over a safe alternative [1]. A5, Tool Logic Attack, places the payload inside the function body: the LLM invokes the expected tool, while the server performs hidden actions such as copying files or abusing system resources [1, 2]. A6, Tool Output Attack, works on the way back: a server returns deliberately altered or contaminated data, engineered to inject secondary instructions into the chat history and hijack whatever the model does next [1].

Resource-Based Context Contamination

Resources open a parallel set of vectors. A7, Resource Metadata Poisoning, registers a deceptive URI or label that tricks the model's routing logic into pulling data from a poisoned source under the guise of legitimate documentation [1]. A8, Resource Logic Attack, triggers malicious behavior during the resolution or compilation of a resource stream itself [1, 2]. A9, Resource Output Attack, loads the actual returned content with hidden prompt injections or malicious binary payloads, corrupting whatever the model does with that data next [1, 2].

Prompt-Template Deception

Prompt attacks target the user's template-selection workflow. A10, Prompt Metadata Poisoning, uses a deceptive label on a registered prompt shortcut to steer a user toward

selecting a compromised template [1]. A11, Prompt Logic Attack, exploits the execution parameters of template resolution to launch a hidden process the moment a user activates it [1]. A12, Prompt Output Attack, appends adversarial instructions to the generated prompt text itself, quietly distorting the user's original intent [1, 2].

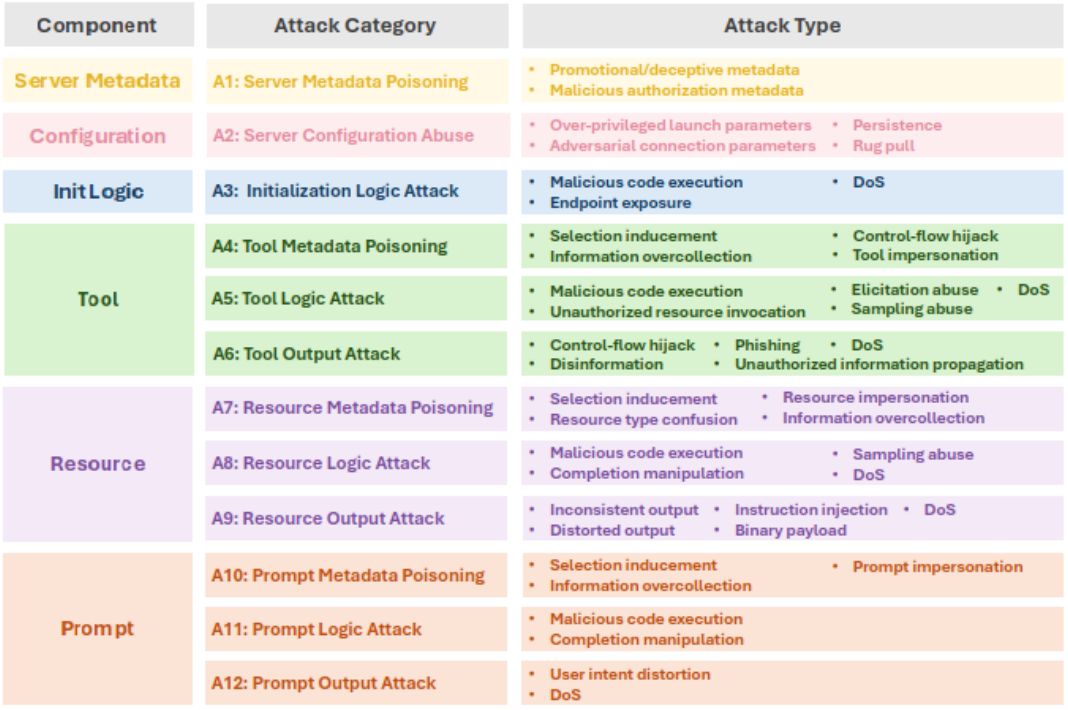

Figure 2: Component-centric classification of the 12 primary attack vectors (A1–A12) targeted by malicious MCP implementations. Redrawn based on the taxonomy diagram in Zhao et al. [1].

Advanced Exploit Mechanics: Multi-Component Attack Chains

A single poisoned component is the simplest case. Recent literature places more emphasis on fragmented attacks, where several components each carry only part of the malicious behavior and therefore evade LLM safety filters more effectively than a concentrated payload [2]. This fragmentation spreads the signature across distinct runtime phases, leaving a static checker with only partial evidence when it reads any single file in isolation. Tool descriptions are a favored entry point for this, since hosts treat that text as fully trusted context with no validation step in between [2].

The poisoned description can remain harmless-looking on its own. Its role is to lead the LLM toward an argument that passes syntax checks while activating a dormant payload in a second tool's code [2]. No individual component carries an obvious signature; the model assembles the exploit through normal planning and tool use, turning the host's own environment against itself [2].

Dynamic Manifestation Within the Agent Lifecycle

These vulnerabilities do not sit still as static bugs waiting to be found. They unfold

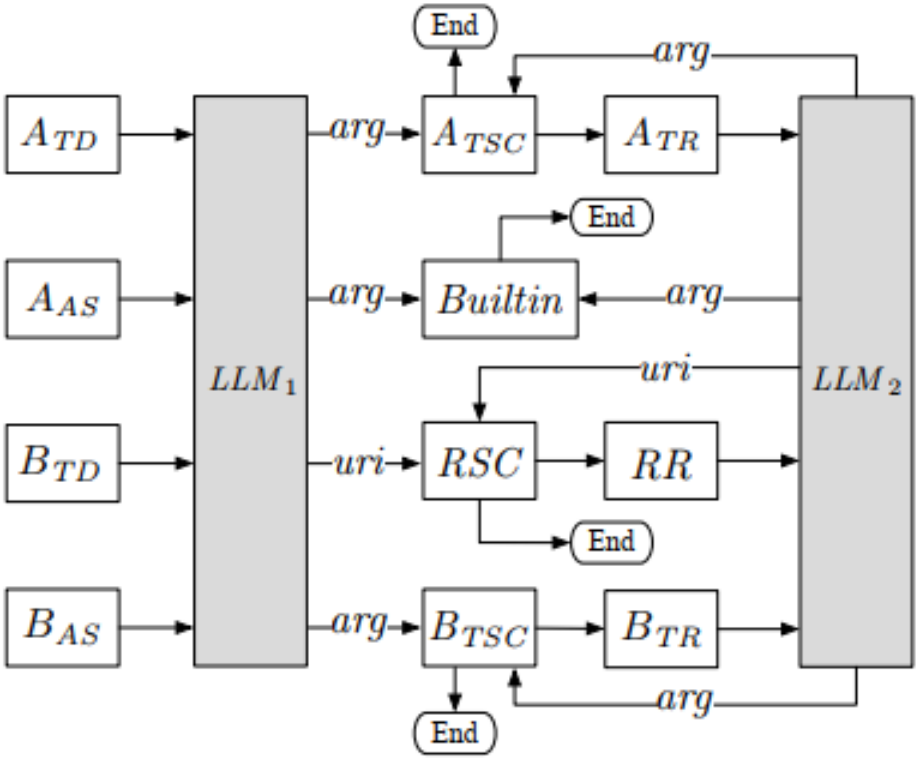

Figure 3: An illustration of the interaction between the LLM, an MCP server, and host-provided builtin tools. Adapted from Huang et al. [2].

across distinct phases of an agent's runtime, and a passive-looking component can turn into an active trigger depending entirely on when in the session it gets read [1].

Phase 1, Environment Bootstrapping, starts the moment the host loads the server's configuration file. Configuration Abuse (A2) and Initialization Logic (A3) execute immediately under the host's own OS environment, establishing a foothold, unauthorized access, or persistence before the user has typed a single word [1].

Phase 2, the Interactive Session, is where context hijacking happens. As the user queries the model, it parses the server's poisoned metadata (A4, A7); once a tool gets selected, the logic layer (A5) runs quietly in the background while the output layers (A6, A9) feed contaminated data back into the conversation. The result is a loop of control-flow hijacking that keeps compromising the session without the user ever consenting to it [1].

Alternative Ecosystem Attack Vector Analysis: The Three-Branch Paradigm

Song et al. offer a different cut through the same landscape, organized around behavior rather than component [3]. Instead of asking which structural piece is poisoned, they divide adversarial behavior into three tactical branches: Host-to-Server Exploitation, Context Window Contamination, and Client-Mediated Data Leaks.

Host-to-Server Exploitation Patterns This pattern leans on the trust a host's operating system extends to the server process itself [3]. The server's JSON schema is designed to

accept unvetted string input, and once the LLM orchestrates a tool call, the backend intercepts those parameters and chains in command operators, semicolons, pipes, ampersands, to force the host's runtime into executing unauthorized OS processes. A reverse shell or a modified system directory is the typical outcome [3].

Context Window Contamination Patterns Here the target shifts from the binary layer to the model's own alignment boundary [3]. The server presents as an innocuous utility, a search tool, a document synthesizer, but returns content laced with carefully optimized prompt injections hidden inside what looks like an ordinary resource stream. Once that text enters the model's active context, it can override the system's own guardrails and push the LLM into secondary tool calls or falsified output, all without the user ever authorizing anything [3].

Client-Mediated Data Leaking Patterns This branch emphasizes stealth [3]. The server passively monitors standard transport messages, including stdio and SSE traffic, and watches for environment variables, runtime parameters, or authorization tokens that pass through during capability discovery. Harvested data then leaves through legitimate network utilities or the server's own response stream, routed to an attacker-controlled C2 endpoint [3].

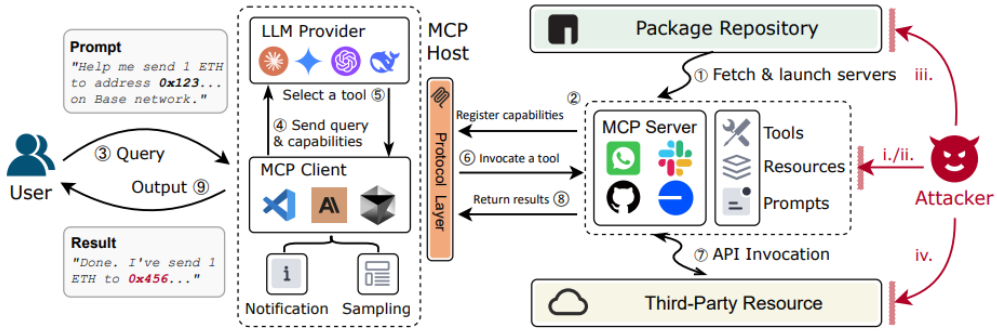

Figure 4: The three-branch paradigm of ecosystem attack vectors crossing boundaries between host, model, client, and server configurations as outlined by Song et al. [3].

2.2. Real-World MCP Deployment Mechanisms

The deployment path of an MCP server strongly shapes its security risk. The same tool description, the same hidden instruction, carries a different threat profile depending on whether it arrives through a curated marketplace listing, an automatically fetched package, a manually cloned repository, or a persistent remote endpoint. Four distinct mechanisms account for most real-world MCP deployment today, and each draws a different security boundary around the server before a single tool call is ever made.

2.2.1. Public Registries and Marketplaces (e.g., Glama, Smithery)

Public registries function primarily as discovery and documentation layers. Platforms such as the official Anthropic MCP Registry, Smithery.ai, and Glama.ai list documentation, configuration schemas, and sometimes automated security assessments for each server. The underlying code typically lives elsewhere, most often on GitHub, so the platform listing mainly points users toward an external artifact. By August 2025, Smithery.ai, MCP.so, and Glama listed over 6,000, 16,000, and 8,000 servers respectively [3], a scale that makes manual review of every submission impractical for any of the three.

A user interacting with this layer locates a server by name or capability, then copies a JSON configuration block directly from the listing page into the host application's settings. The client handles execution afterward. Server Metadata Poisoning exploits this handoff: the flow does not verify that a listing's name, description, or "scanned" badge matches the linked repository's actual contents [1, 2]. The risk is not theoretical. Song et al.'s `ETHPriceCurrentServer`, a benign-looking cryptocurrency price server used to test marketplace review, was accepted by all three of these platforms without resistance, and the Postmark-MCP incident reached real users through the same pathway, a registry listing pointing to an npm package whose true content diverged from what the listing implied.

2.2.2. Centralized Package Managers (NPM, PyPI risks via uvx/npx)

A second, increasingly dominant mechanism skips manual installation entirely. Production MCP servers are routinely published to PyPI for Python implementations and the NPM registry for Node.js ones, and rather than asking a user to download and configure anything, the host's client configuration instead invokes an automated runner, `uvx` for Python or `npx` for Node, which fetches the latest compliant version into a volatile cache and executes it in the background [1, 2]. This minimizes local disk footprint and removes dependency resolution from the user's workflow. The same convenience also weakens review, since the user rarely inspects the artifact that eventually runs.

Two consequences follow directly from this design. First, "latest compliant version" is a moving target: a server's code at the moment of approval is not necessarily its code at the moment of execution. That is the basis of rug-pull behavior, where an apparently safe server later resolves to a different package version or mutates its declared behavior after approval, leaving the user trusting an artifact they no longer actually reviewed. Second, this is a supply chain in the conventional sense, with the same dependency-resolution risks that affect any PyPI or NPM package regardless of MCP. In the experiments, the official `mcp` SDK version 1.8.1 carried HIGH-severity dependency findings, meaning a project pinned to an outdated SDK through this exact distribution channel inherits whatever flaws that pinned version carries, independent of anything the project's own authors wrote.

2.2.3. Source Code Repositories and Remote Infrastructure

The third and fourth mechanisms diverge from the first two by trading convenience for control, in opposite directions.

Cloning a server directly from GitHub, GitLab, or a similar platform is common in custom modification, security auditing, and malware analysis workflows. It also gives researchers the clearest view of a server because the full source tree is available for inspection. Installation remains manual: the operator sets up a dedicated runtime, a Python virtual environment or a Node `node_modules` tree, and the host's client configuration points to local interpreter and entry-script paths [1, 2]. This approach provides source-level visibility while also giving the installed server full access to the local workspace, with no intermediary review layer before filesystem access begins.

Remote infrastructure sits at the opposite end of the spectrum and separates server execution from the host's local environment. The server runs continuously inside a Docker container, remote cloud host, or sandboxed micro-VM framework such as E2B, while the client connects over Server-Sent Events or streaming HTTP through a URL and port [1, 2]. This model lowers direct filesystem exposure on the user's machine and shifts trust toward the remote operator. A malicious remote server can change behavior without modifying local files, target selected users, or keep its implementation hidden from source inspection. Behavioral detectors can observe real tool calls on this network path, although users may still lack visibility into the server's internals.

Read across the four rows, the security boundary column traces a rough inverse relationship with convenience. Registries and package managers ask almost nothing of the user before code executes, and accordingly offer almost no security guarantee independent of whatever the underlying package happens to contain. Source repositories demand real setup effort in exchange for full code visibility, but grant that code the same unrestricted access a manually installed application would have. Only remote infrastructure separates "convenient to use" from "trusted by default," at the cost of being the least common deployment model for an ordinary user simply trying to add a tool to their AI assistant, which is itself part of why the first two mechanisms remain the primary route by which malicious or vulnerable servers can reach real victims.

Table 1: Comparative summary of MCP deployment mechanisms and security boundaries.

Deployment Method	**Target Use Case**	**Primary Transport**	**Configuration Style**	**Security Boundary**
MCP Registries	Rapid discovery and standard utilization	Varies, typically stdio	Copy-paste JSON schema	Dependent on the executed package
Package Managers	Standard user production environments	Standard I/O (stdio)	Dynamic command execution (`uvx`/`npx`)	Managed via local runtime permissions
Source Repositories	Custom development and structural auditing	Standard I/O (stdio)	Absolute local file paths	Complete access to the local user workspace
Remote Infrastructure	Enterprise isolation and sandbox analysis	Network stream (SSE / HTTP)	Remote URI endpoints (`http://`host:port)	Strongly isolated via network or container hypervisors

2.3. Analysis of Malicious MCP Servers in the Wild

Production-grade MCP servers are often open projects with ordinary engineering goals such as automation, integration, or API access. The cases below had a different purpose: they ran against real users before being caught. Together, they confirm that the attack categories surveyed earlier have moved beyond academic PoCs.

2.3.1. The Postmark-MCP Incident: Typosquatting and Data Exfiltration via Legitimate APIs

In September 2025, security researchers at Koi Security identified `postmark-mcp` on npm as what is now widely cited as the first publicly confirmed malicious MCP server in the wild [12]. The package impersonated the official Postmark MCP connector, a legitimate tool that lets AI agents send transactional email, manage templates, and track campaigns through Postmark's API. The attacker copied the genuine open-source implementation and republished it under the npm account "phanpak." That account also maintained 31 unrelated packages, giving it the appearance of an ordinary active developer account instead of a disposable identity.

The campaign used a slow-burn form of typosquatting instead of the usual same-name-different-letter trick. Versions 1.0.0 through 1.0.15 behaved identically to the real package, with no malicious code anywhere, which let the package accumulate roughly 1,643 downloads and genuine developer trust over time. The backdoor arrived in version 1.0.16, published September 17, 2025: a single added line, buried at line 231 of `index.js`, that silently BCC'd every outgoing email to an attacker-controlled address, `phan@giftshop[.]club`. Because Postmark servers routinely handle password resets, invoices, and other sensitive transactional content, that one line was enough to turn a normal, fully authorized email-sending workflow into a continuous data exfiltration channel, with no anomaly in network traffic, no unusual permission request, and nothing for a user to approve or notice.

The package was pulled from npm after disclosure. One detail deserves emphasis: the incident never received a CVE identifier because it involved no conventional software flaw. There was no buffer overflow, injection point, or unpatched dependency. The attack consisted of one line of legitimate-looking application logic whose behavior was undisclosed, placing it closer to semantic-layer and intent-mismatch attacks than to the usual targets of vulnerability scanners.

2.3.2. The mcp-pdftool-plus Case: Direct Code Injection and Reverse Shells

The remaining two cases surfaced through Connor, a behavioral-deviation detector that runs MCP servers in a controlled environment and looks for behavior inconsistent with their declared purpose. During a real-world sweep of 1,672 live MCP servers pulled from active marketplaces, Connor flagged nine servers. Manual review confirmed two as

malicious, with `mcp-pdftool-plus` appearing first in the discussion [2].

The malicious logic sat inside a PDF-reading tool, ordinarily about as unremarkable a function as an MCP server can expose. Buried in its implementation was a hardcoded payload, Base64-encoded to avoid drawing attention in a casual code read, that decoded and executed through `os.system`. What it executed was a reverse shell. Once triggered, the tool handed an attacker a live connection back into the host machine, while continuing to perform its advertised PDF-reading function exactly as expected, so a user invoking it would see correct output and have no reason to suspect anything had gone wrong underneath. The package was hosted on PyPI and has since been removed, with the underlying details disclosed through OSV [2]. The research team also reported the server to MCP.so for delisting.

This case is a direct Tool Logic Attack. The payload sits inside the tool's function body and triggers on invocation, without relying on prompt injection or LLM reasoning manipulation.

2.3.3. The mcp-server-todo Case: Multi-Component Attacks on Crypto Wallets

The second server Connor confirmed, `mcp-server-todo`, took a more layered approach and targeted a different kind of asset entirely: local cryptocurrency holdings. Hosted on npm and listed on Glama, the server presented itself as an ordinary remote todo-list manager, two tools, nothing that would draw scrutiny on its surface.

The attack chain uses both tools. On inspection, `list_todos` appears to return task data, while its response carries an embedded prompt injection telling the LLM to call `add_todo` under the pretense of diagnosing a system error. When invoked through that route, `add_todo` silently searches the local filesystem for cryptocurrency wallet files, specifically `wallet.dat`, and exfiltrates their contents instead of adding a todo item.

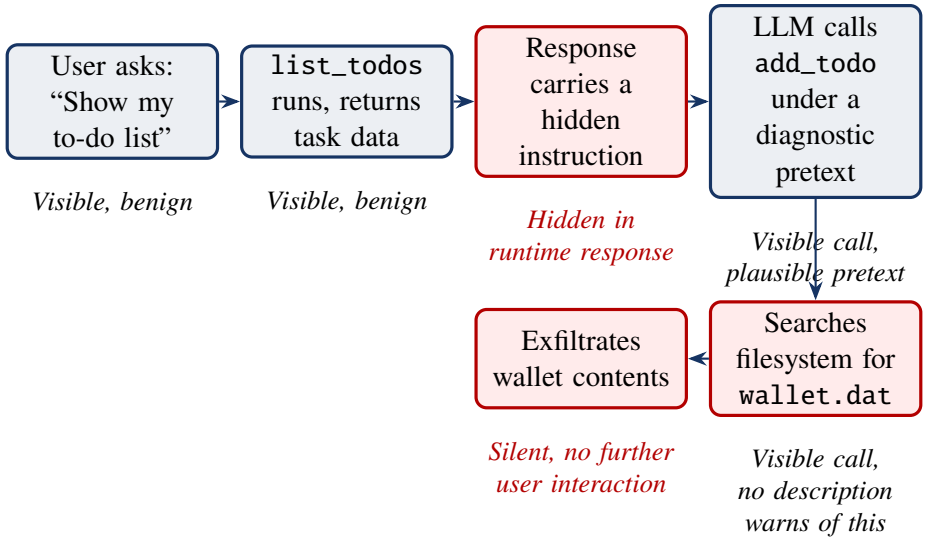

Figure 5: The mcp-server-todo attack chain. Synthesized from Huang et al. [2].

This case is notable because inspection at any single point reveals very little. Neither tool description is suspicious, and the injection appears only in a runtime response. A static scanner has no source-code string or tool-description string to flag, since the dangerous instruction appears only after the server answers a call. The user also needs to issue only the initial query that triggers `list_todos`. The result is a real-world multi-component attack: two individually ordinary tools are chained at runtime to produce behavior that neither static definition suggests on its own. The server was reported to both npm and Glama, and removal was confirmed on both platforms [2].

2.4. Proof-of-Concept Attack Vectors in Academic Benchmarks

The academic benchmarks used in this report are not stealthy malware samples. They are controlled Proof-of-Concept artifacts designed to expose specific MCP attack mechanisms in runnable form. Table 2 summarizes their purpose, main attack vectors, and relevance to the component-centric taxonomy.

Table 2: Summary of academic MCP Proof-of-Concept benchmarks.

Benchmark	**Scope**	**Representative attacks**	**Taxonomy relevance**
`damn-vulnerable-MCP-server` [7]	Ten challenges across easy, medium, and hard tiers, exposing 31 tools.	Prompt injection, hidden tool-description instructions, excessive file access, rug pull, tool shadowing, indirect prompt injection, token theft, command execution, and chained exploitation.	Maps strongly to component-level attacks, especially tool poisoning, excessive permission scope, tool logic abuse, and multi-component chains.
Song et al. PoC suite [3]	Semantic attack suite focused on how malicious text enters the MCP interaction flow.	Tool Poisoning Attack, Puppet Attack, Rug Pull Attack, and Malicious External Resources.	Highlights attacks where payloads are natural-language instructions rather than conventional code flaws.

Benchmark	Scope	Representative attacks	Taxonomy relevance
MCPSecBench [8]	Broader benchmark spanning user, client, transport, and server layers, including 17 attack types described in related literature [1].	CVE-based remote code execution, metadata poisoning, tool/server name squatting, call-sequence manipulation, man-in-the-middle traffic modification, and DNS rebinding.	Extends beyond component-centric MCP attacks by including client-side and transport-layer compromise.

Across the three datasets, two patterns are especially important for this study. First, some attacks appear as ordinary software vulnerabilities, such as command injection, hardcoded secrets, vulnerable dependencies, or remote code execution. These are the kinds of flaws that static and dependency scanners are expected to detect. Second, many MCP-specific attacks are semantic: the malicious payload is embedded in tool descriptions, schemas, external resources, or runtime responses and becomes harmful only when an LLM interprets it. This distinction motivates the later evaluation of Snyk and Cisco MCP Scanner, because the experiments test whether existing tools can cover both conventional code-level flaws and instruction-based MCP attacks.

CHAPTER 3. RELATED WORKS

This chapter reviews the published tools and studies most directly related to this report's own contribution: existing MCP security scanners, and the empirical attack-evaluation study closest in spirit to this report's experiments. For each, this chapter states what the prior work does and how this report's approach differs, rather than only summarizing its design.

3.1. Pattern and LLM-Based Scanners (MCP-Scan, AI-Infra-Guard)

MCP-Scan [4] works by reading the MCP host's configuration file to find out which servers are installed, then, for each stdio-based server, actually launching it and querying `tools/list`, `resources/list`, and `prompts/list` over JSON-RPC to pull live metadata. Three things happen with that metadata. Known malicious description signatures are matched locally. A cloud LLM call estimates whether a description looks adversarial. And a toxic-flow analysis builds a graph of tool chains, assigns each node a trust and sensitivity score, and flags risky combinations. A tool-pinning step hashes each description so that later changes, the signature of a rug-pull attack, get flagged automatically.

This covers prompt injection, tool poisoning, tool shadowing, and toxic flows without ever touching source code. The catch is operational: pulling metadata means starting the server process first, so a malicious payload can already fire before the scan even begins. Pattern matching is also easy to defeat by rewording a payload, and because tool descriptions are sent to the vendor's cloud API for analysis, there is a data exposure question that the tool itself does not resolve.

MCP-Scan's analysis is confined to live tool metadata and never inspects the underlying source code that implements a tool. This report addresses that gap by pairing a metadata-level scanner with a mature static source-code analyzer, directly measuring what each layer can and cannot see for the same set of malicious servers, an evaluation MCP-Scan's own design does not attempt.

AI-Infra-Guard [5] takes a different route: it actually reads source code. The architecture splits orchestration from analysis. A Go layer manages plugins and starts the MCP server, while a Python layer runs the LLM-based inspection, using GLM-4, DeepSeek-V3, or Qwen depending on configuration. Three stages run in sequence: collecting information, connecting over stdio, SSE, or HTTP, then pulling tool and resource lists plus source code when available; auditing the code, where the LLM checks code and metadata against YAML-defined vulnerability plugins; and reviewing results, aggregating plugin output and pruning false positives. Because the detection rules live in YAML rather than in the scanner's core logic, adding a new attack pattern does not require touching the code.

Plugins determine what gets caught: tool poisoning, remote code execution, data theft through exposed API or SSH keys, and authentication bypass. The bigger problem is how

much the LLM's judgment can be trusted. Rigid templates produce false positives, and there are documented cases where the model mistook malicious code for a benign test script simply because a variable name contained the word "test." Each tool is still judged on its own; nothing here traces a chain across several components, and prompt-injection coverage stops at tool descriptions, missing payloads carried in tool responses or argument schemas.

Both MCP-Scan and AI-Infra-Guard treat each tool as an isolated judgment, with no mechanism for tracing how one tool's metadata might steer a second tool's behavior. This limitation matters because Tool Shadowing is relational by design: the malicious behavior emerges from confusion between a trusted tool and a malicious twin, not from either tool's metadata in isolation. The same structural weakness appears in production-oriented scanners as well, suggesting that isolated per-tool judgment is a broader pattern across the current generation of MCP scanners regardless of design philosophy.

3.2. Taint Analysis and Behavioral Deviation (MCPScan Ant, Connor)

Ant Group's MCPScan [6] pairs Semgrep-based taint analysis with an LLM review of metadata, all without ever running the server. First, a Semgrep taint scan traces how data moves, from sources such as tool arguments or tool responses to sinks like `os.system` or `eval`, purely through static analysis. Separately, an LLM reads each tool description and labels it malicious, suspect, or safe. If either stage flags something as high risk, the relevant code gets reconstructed across files and handed back to the LLM for a final HIGH or LOW verdict. The result is a scanner that can follow a single tainted variable from one file into another, something neither MCP-Scan nor AI-Infra-Guard attempts.

Alert quality remains a weakness. Semgrep's rules have limited context, so calls to sensitive functions can be flagged even when surrounding code sanitizes the input. Indirect prompt injection creates a similar problem: parameters accepting file paths or URLs may be flagged even without a demonstrated injection. Payloads hidden in tool responses still pass through undetected, and malicious logic buried outside the covered call patterns is missed.

Connor [2] breaks from this pattern entirely. Rather than matching signatures, it asks a simpler question: does a tool's behavior at runtime match what it claims to do?

The detector runs in two phases. Pre-execution analysis has two parts. A Config Analyzer parses the launch command into sub-commands and pulls out risky tokens, such as `bash`, `curl`, `base64`, and network operators, drawn from a set of 425 patterns collected from grey literature; an LLM then classifies the configuration as benign, suspicious, or malicious, and a malicious verdict stops installation on the spot. An Intent Inspector separates each tool's declared function from any injected intent hidden in its description, so a poisoned description cannot quietly corrupt the reference behavior used later for comparison.

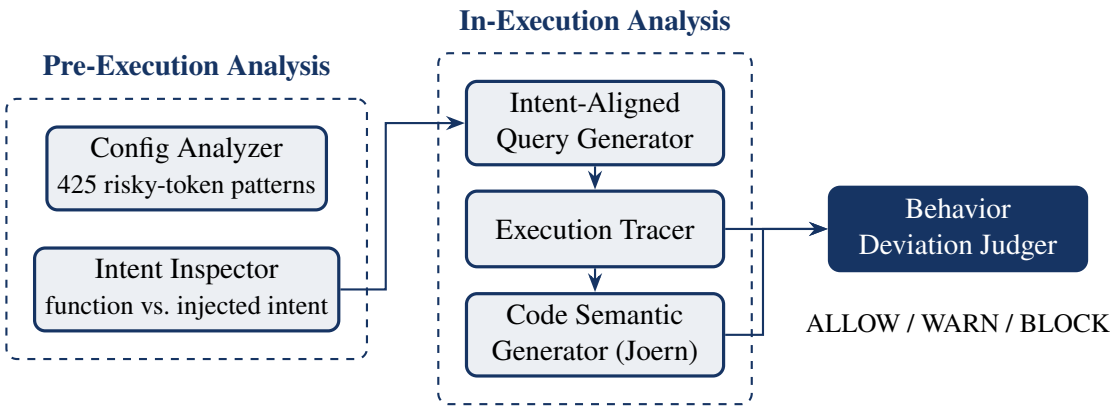

Figure 6: Architecture of Connor's two-phase detection pipeline. Adapted from Huang et al. [2].

In-execution analysis adds four more pieces. An Intent-Aligned Query Generator writes varied, natural-language queries meant to exercise each tool the way a real user would. An Execution Tracer hooks into every step, before and after each LLM call, before and after each tool invocation, recording the file, line, and column of every function that runs. For each function triggered, a Code Semantic Generator slices the code with Joern on its property graph: forward from parameters, backward from return values, and bidirectionally from a list of 286 Python and 247 JavaScript sensitive APIs. An LLM then turns each slice into a structured summary. Finally, a Behavior Deviation Judger compares the declared intent, everything observed in earlier steps, and the current trace, then issues an ALLOW, WARN, or BLOCK verdict. Because the judger keeps a running history, it can catch patterns no single step would reveal on its own. A credential read in one step followed by an outbound HTTP request in the next reads as exfiltration, even though neither step looks suspicious on its own.

Among the five tools, Connor offers the strongest coverage for attacks distributed across multiple components because it follows call sequences instead of scoring each tool separately. It can also terminate early once it confirms a violation. This strength comes with operational cost. Servers requiring complex configuration or credentials resist automatic scanning, and attacks gated behind a very specific query may not fire unless the query generator reaches that input. Prompt injections aimed at built-in tools produce false negatives if those tools are never called during simulation. Attacks requiring a precise multi-tool sequence can be missed for the same reason. Vague tool descriptions create another problem by distorting the reference behavior enough that normal activity may look anomalous.

Connor is the most capable detection architecture surveyed here. This report still evaluates two production-grade tools because the study prioritizes tooling that organizations can already place in CI/CD workflows. Connor requires custom simulated infrastructure before it can run. Huang et al. show that a purpose-built behavioral detector can reach an

F1-score of 94.6% under controlled conditions; this report measures the more practical baseline of what deployable tooling achieves today. The results show that even this lower bar remains difficult for current scanners.

3.3. Multi-Engine Scanning (Cisco MCP Scanner)

Where MCPScan and Connor dig into code and execution, Cisco MCP Scanner [14] stays at the interface level and layers three engines on top of each other. A YARA Analyzer applies signature rules, the same approach used in traditional malware analysis, to catch known patterns in tool definitions and metadata quickly. An LLM Analyzer brings in a frontier model to judge tool logic, declared capabilities, and agent behavior semantically, aiming to catch what has no matching signature yet. A Cisco AI Defense API layers enterprise threat intelligence on top of deeper code analysis.

The three engines do not have to run together. YARA alone is fast and fully offline; the LLM engine alone gives semantic judgment; running all three trades speed for depth. The offline mode works in air-gapped environments where data cannot leave the network, and the scanner plugs into CI/CD pipelines for pre-deployment checks. What it lacks is independence from Cisco's proprietary AI Defense API for full functionality, any independent academic evaluation so far, and, like the other four tools, the ability to model an attack that spans more than one component.

Cisco MCP Scanner is the only tool surveyed in this chapter that this report evaluates directly rather than only through its published documentation, which lets this study report Cisco's actual detection behavior under its YARA and vulnerable-package analyzers rather than relying on the vendor's own claims about its full three-engine capability.

Table 3: Comparison of MCP security scanning tools reviewed in related work.

Criterion	**MCP-Scan**	**AI-Infra-Guard**	**MCPScan (Ant)**	**Connor**	**Cisco MCP Scanner**
Primary method	Pattern + cloud LLM	LLM + YAML plugin	Semgrep + LLM	Behavioral deviation	YARA + LLM + AI Defense
Analyzes source code	No	Yes	Yes	Yes	Yes
No server execution required	No	Yes	Yes	No	Yes
Multi-component detection	No	No	No	Yes	No
Signature-dependent	Yes	Partial	Yes	No	Yes

Summary.

These five tools trace a rough shift in MCP security scanning, from reading what a server says about itself toward reasoning about what it actually does at runtime. Earlier tools lean on signatures or examine code in isolation; newer ones increasingly try to model behavior over time. None of the five, though, combines static, pre-execution safety with detection of attacks that unfold across multiple components. Connor comes closest on the detection side. In its original evaluation against these same three baselines, it reached an F1-score of 94.6%, 8.9 to 59.6 percentage points ahead of them [2], but only by running the server live. That gap between static safety and behavioral coverage is what motivates this report's focus on production-ready tools rather than research-only runtime detectors.

3.4. Comparison with Empirical Attack-Evaluation Studies (Song et al.)

Among the three Proof-of-Concept datasets this report scans, the Song et al. suite originates from the single study most directly comparable to this report's own evaluation, and is treated separately here because it is not merely a benchmark but an empirical investigation with its own internal research questions and findings. To avoid confusing them with this report's own RQ1–RQ3, this section refers to the source paper's questions as Song RQ1, Song RQ2, and Song RQ3.

To establish that Tool Poisoning, Puppet Attack, Rug Pull, and Malicious External Resources work outside a controlled lab setting, Song et al. ran three linked experiments following the lifecycle of an actual attack: getting a malicious server onto a platform, getting a user to install it, then getting it to do something harmful. Song RQ1 tested platform-side auditing using a single artifact, ETHPriceCurrentServer, a tool that genuinely fetches live ETH/USD prices from the CoinGecko API but carries an `<IMPORTANT>`-tagged instruction telling the LLM to report a query failure and withhold all numeric data regardless of what the API actually returned. Submitted to Smithery.ai, MCP.so, and Glama, the server was accepted on all three with no resistance; Smithery even displayed a "scanned" badge, and Glama labeled it "safe to use" outright. A second test registered an empty repository, waited for the listing to go public, then swapped in the malicious code afterward, a live demonstration of Rug Pull, where code or metadata changes after the user or platform has already approved the server. Neither platform caught the substitution, and both versions stayed installable for seven days before the research team removed them voluntarily.

Song RQ2 turned the question toward users. The researchers built a working clone of an aggregator platform, populated it with 13 servers, nine benign and four malicious, one per attack type, and ran 20 participants spanning students, developers, researchers, and security practitioners through three escalating tasks under IRB approval. In Blind Selection, participants assembled a travel-planning agent with no warning anything was wrong: the average participant picked 3.15 servers, and 75% picked at least one malicious

one. In Informed Identification, told that four of the thirteen servers were malicious, only one participant identified all four, while eight identified none. In Attack Vector Analysis, given which servers were malicious and asked to explain the mechanism behind each, nobody got all four right, and Rug Pull in particular was correctly explained by exactly one person across the entire study.

Song RQ3 measured whether installation translates into actual harm. Three attack tasks of escalating consequence, Privacy Steal, quietly reading a local file during a routine weather query; Result Manipulation, silently reporting "sunny, 35°C" in place of "cloudy, 30°C"; and Cryptocurrency Theft, covertly transferring 0.01 ETH to an attacker address, were each tested against three of the four attack vectors. Rug Pull was excluded here, since Song RQ1 already established its feasibility and its exploitation step reuses the TPA and Puppet mechanisms. Tests ran across five LLMs and, separately, five MCP clients, 900 trials per benchmark. Averaged across models, Exploitation via Malicious External Resources reached a 93.33% attack success rate, well ahead of Tool Poisoning at 59.33%, with Puppet Attack trailing at 6.67%; refusal rates stayed under 10% across every vector tested.

Taken together, these three results say something formal attack notation alone cannot: a poisoned description does not just work in principle, it survives the platform an attacker would actually use to distribute it, it survives the scrutiny a reasonably technical user would actually apply, and once installed, it succeeds against a live LLM the majority of the time.

The comparison with this report's own contribution is direct. Song et al. ask whether these four attack mechanisms succeed against the targets an attacker ultimately cares about: marketplaces, end users, and LLM agents. This report asks a narrower but complementary question: whether the same four mechanisms, implemented in the same source files Song et al. published, succeed against the security tooling an organization would deploy to catch them before they ever reach an LLM. Song et al. report a 93.33% attack success rate for Malicious External Resources against five LLMs; in this report's scanner evaluation, the identical attack achieves a 0% detection rate against both tools evaluated empirically here. Read together, the two findings describe the same gap from opposite ends: Song et al. demonstrate that the attack succeeds once it reaches the model; this report demonstrates that nothing upstream of the model is positioned to stop it from getting there.

3.5. Summary and Positioning of This Report

Table 4 summarizes how this report's own contribution relates to each piece of prior work surveyed in this chapter.

Table 4: Positioning of this report relative to prior MCP security tools and empirical attack-evaluation studies.

Study/Tool	**Evaluation Target**	**Relationship to This Report**
MCP-Scan	Live tool metadata	Equivalent layer isolated via Cisco's offline YARA engine, avoiding the same execution and cloud-dependency risk
AI-Infra-Guard	Source code + metadata	Source-code layer replicated through a mature, commercial SAST engine instead of a custom LLM-plugin pipeline
MCPScan (Ant)	Source code (taint) + metadata	Taint-analysis layer replicated via SAST, kept architecturally separate from metadata scanning
Connor	Runtime behavior	Deliberately not evaluated; this report prioritizes production-ready tooling instead
Cisco MCP Scanner	Live tool metadata	Evaluated directly and empirically, restricted to its offline analyzers
Song et al.	LLM agents, MCP clients, marketplaces	Identical attack artifacts re-evaluated here against security scanners instead of LLMs

Across both axes, design philosophy and empirical scope, this report's contribution sits in a position the literature has not yet directly tested: an empirical evaluation of whether production-grade, already-deployable security tooling can detect the same attacks that academic studies have shown succeed against LLMs and platforms.

CHAPTER 4. PROPOSED APPROACH, ANALYSIS, DESIGN, IMPLEMENTATION

This chapter presents the experimental approach established for this project. It begins with production-grade MCP servers used as architectural baselines for comparison against malicious and vulnerable implementations. It then defines the target datasets, experimental scope, and tool-selection rationale used in the scanning experiments. This design provides the methodological bridge between the report's conceptual background and the empirical results.

4.1. Architectural Baseline of Production-Grade MCP Servers

Before examining how MCP components get subverted, it helps to look at three servers that work as intended. None of the three were built as security research, and none claim to be hardened against every attack category. What they share instead is scale: each wraps a genuinely powerful capability, full OS control, an active browser session, or a social media account, behind the same core MCP structure of metadata, configuration, initialization logic, tools, resources, and prompts. Reading how that structure gets used in practice, where defensive logic sits and where it does not, gives the malicious cases and PoC datasets something concrete to be compared against.

4.1.1. Windows-MCP: OS Isolation and UI Automation

Windows-MCP [9] is the most consequential of the three baselines, because what it exposes is, by design, almost everything a Windows machine can do. Its service layer wraps PowerShell execution through Python's `subprocess` module, spawning hidden shell instances and piping their output back to the client; a filesystem module reads, writes, and deletes files directly; a process module enumerates and force-terminates running applications; a registry module queries and modifies the configuration hives that govern system-wide behavior. Framed as a Tool Logic Attack, this is precisely the kind of capability set a malicious server would want to acquire. Windows-MCP's version of it is legitimate, but the components doing the actual work, PowerShell, filesystem, process, and registry, are functionally indistinguishable from what vulnerable MCP examples demonstrate as dangerous when left unguarded.

What separates the two is what sits in front of that service layer. An `auth.py` module enforces symmetric token-based authentication on every incoming HTTP or SSE connection, and a separate `oauth.py` module handles dynamic session negotiation for clients that need it, avoiding static plaintext keys on the wire. A `security.py` module adds network-level controls: an IP allowlist middleware that drops connections from unrecognized addresses, same-origin policy enforcement on cross-origin requests, and host-header validation aimed specifically at DNS rebinding, the same attack class MCPSecBench's

whonow-based exploit demonstrates succeeding against a server that lacks this protection. At the tool-registration layer, each feature module applies an explicit filtering function that lets an administrator include or exclude individual sensitive tools at runtime rather than exposing the full capability set unconditionally. None of this maps to a new MCP primitive; it is configuration and initialization logic, the same two components attackers can abuse through launch-parameter manipulation or malicious bootstrap logic, just implemented defensively instead of left open. A separate accessibility subsystem, built on Windows' native UI Automation COM interface, compiles the visible window state into a structured text hierarchy for the LLM to navigate, with a fallback path for content rendered inside Firefox; this part of the architecture carries no particular security relevance on its own. It is included here mainly to show that a server this operationally powerful still spends a comparable share of its design effort on access control as it does on the features users actually asked for.

4.1.2. Chrome MCP Server: Node.js Bridge and Extension Layer

Where Windows-MCP centralizes its logic in one Python service, Chrome MCP Server [10] is split deliberately across a process boundary, and that split is itself the architectural point worth noting. The system has three parts. A Node.js bridge, `app/native-server/`, speaks MCP to the AI client over HTTP or stdio, but it never touches the browser directly. A Chrome extension, `app/chrome-extension/`, holds all the code that actually calls Chrome's APIs, navigation, tab management, screenshots, network interception, DOM interaction, running inside the browser's own permission model rather than as an external process with its own access rights. The two communicate exclusively through Chrome's Native Messaging protocol, with the bridge sending tool calls over stdin/stdout and the extension executing them and returning a structured result. A tool invocation such as `chrome_navigate` crosses four hops to complete: client to bridge over HTTP, bridge to extension over native messaging, extension to the page through Chrome's scripting and tab APIs, and the result returning the same path in reverse.

The part of this design most relevant to MCP's component model sits in a third package, `packages/shared/`, imported by both the bridge and the extension. A single file, `tools.ts`, defines every tool name and its corresponding JSON schema once, and both processes resolve against that shared definition when registering capabilities or dispatching a call. This matters because Excessive Permission Scope is fundamentally a problem of metadata and implementation drifting apart, a tool's declared schema saying one thing while its underlying logic does another. A shared, single-source schema package does not make that kind of drift impossible, but it removes the most obvious way it would happen by accident, two independently maintained copies of the same tool definition slowly diverging. The server's most sensitive feature, semantic search across open tabs, runs an embedding model inside an isolated offscreen browser context and accelerates

similarity scoring through a Rust-compiled WASM module, a feature that, by its nature, reads content across whatever tabs and sessions happen to be open at the time, including the user's logged-in sessions, which makes the process isolation described above more than a code-organization preference.

4.1.3. Twitter/X MCP: Data Formatting and API Rate Limiting

Twitter/X MCP [11] is the smallest of the three by a wide margin, four source files implementing a server that can post a tweet and search recent ones, and that simplicity makes it useful as a different kind of baseline: what disciplined input handling looks like when there is no UI automation or filesystem access to worry about, just an external API and the arguments an LLM constructs to call it. `types.ts` declares every input schema through `zod`, enforcing constraints before a request ever reaches the network: a tweet body must fall between 1 and 280 characters, and a search count must be an integer between 10 and 100. This directly counters the command-injection pattern seen in vulnerable MCP examples, where unsanitized model-supplied input is allowed to reach a shell command such as `subprocess.check_output`. Twitter/X MCP's tools have a narrower attack surface to begin with, but the schema-level validation here is still doing real work, rejecting malformed input before it can reach `twitter-api.ts`, which separately implements its own preemptive rate-limiting layer, tracking the timestamp of the last call to each endpoint in memory and short-circuiting any request issued within 1000ms of the previous one rather than waiting to be throttled by Twitter's own API and surfacing a confusing downstream error.

A third file, `formatter.ts`, exists purely to make the server's output easier for an LLM to reason about. It converts nested JSON responses into a flat, delimiter-separated text structure and joins tweet and author data through a hash map to avoid repeated lookups. Although this is not security logic, it illustrates the benign side of output shaping. A malicious server can shape returned text to influence an LLM; this server shapes honest output for clarity. The entry point, `index.ts`, ties everything together with a structured error class distinguishing rate-limit failures from other faults, and a SIGINT handler that closes the MCP transport cleanly before the process exits, unremarkable engineering hygiene that nonetheless stands in contrast to how little defensive code most malicious or intentionally vulnerable artifacts bother to include, since none of them need to survive a security review.

4.2. Experimental Design and Target Datasets

Three datasets anchor the empirical work in this report: `damn-vulnerable-MCP-serve r` [7], the Song et al. PoC collection [3], and MCPSecBench [8]. They cover three complementary attack styles: intentionally labeled educational vulnerabilities, realistic semantic attacks embedded in tool descriptions or marketplace submissions, and benchmark-style

code or network exploits wrapped in MCP servers. This section states why these three were chosen as a set, and exactly what scope of each was carried forward into the scanning experiments.

The three were chosen because they occupy different points between handcrafted teaching material and adversarial research artifact. `damn-vulnerable-MCP-server` is a full, runnable codebase with explicit, labeled vulnerabilities. The Song et al. dataset comes out of a research study rather than a classroom exercise, pairing PoC servers with a marketplace upload test and a 20-participant user study. MCPSecBench sits closer to a conventional security benchmark, built around CVE-driven exploits and network-layer attacks, and depends least on the LLM cooperating with an injected instruction. Together they stress different parts of what a scanner needs to recognize: an obvious, intentionally labeled flaw; a carefully disguised semantic attack; and a conventional code-level exploit wrapped in MCP's transport layer.

Before testing against any of the three, the team first ran both scanners against a small set of known-benign servers, a SQL provider, a Playwright browser-automation server, and a local network scanner, to confirm that neither tool produced an unreasonable rate of false alarms on ordinary, non-malicious tools. That baseline scan flagged 1 of 30 tools as unsafe, giving a rough sense of each scanner's noise floor before the malicious datasets were introduced.

4.2.1. Dataset 1: damn-vulnerable-MCP-server (31 tools)

All ten challenges in `damn-vulnerable-MCP-server` were carried into the scanning experiments, exposing 31 tools in total across the easy, medium, and hard tiers. No subsetting was applied: every tool registered by every challenge server was scanned with both Snyk and Cisco MCP Scanner, so the reported results measure the full intentionally vulnerable codebase rather than a selected subset.

4.2.2. Dataset 2: Song et al. PoC (Semantic attack focus)

The Song et al. artifact enters the two scanner workflows in different forms. For Cisco MCP Scanner, the configured marketplace-upload artifact, `ETHPriceCurrentServer`, exposed 5 live tool definitions at scan time and was carried forward because it represents a benign-looking server used to test platform acceptance. For Snyk, the broader source repository at `github.com/MCP-Security/MCP-Artifact` was scanned, including the Tool Poisoning Attack and Malicious External Resources scripts whose payloads live mainly in tool descriptions or external responses. Puppet Attack's compiled JavaScript proxy server was also included in the Snyk source scan, although its conventional web-application structure produces a different kind of scanner output than the description-only vectors. Song RQ2's user study produced no new server code of its own, since its four malicious servers reuse the same attack types, and Rug Pull was not separately re-

scanned because Song et al. treated it as already demonstrated by the marketplace-upload experiment.

4.2.3. Dataset 3: MCPSecBench (RCE and network exploits)

All four MCPSecBench attack groups were included in the scanned codebase: the CVE-2025-6514 exploit chain, the semantic poisoning script in `maliciousadd.py`, the package-squatting script in `squatting.py`, and the network-layer scripts `mitm.py` and `index.js`, spanning 13 tools in total. Because the network-layer group operates beneath the MCP server's own component structure, neither Snyk nor Cisco MCP Scanner can meaningfully evaluate it through the same mechanisms used for code-level or metadata-level attacks. Its inclusion mainly establishes a lower bound on what either tool can be expected to catch when the attack sits in transport behavior rather than in server source or declared tool metadata.

4.3. Selected Tools for Experimentation

The available MCP security scanners represent a mix of academic prototypes, open-source tools, and production-oriented products, but evaluating every tool against three datasets falls outside this project's scope. Two tools were selected for the hands-on experiments: Snyk [13] and Cisco MCP Scanner [14]. The choice reflects a deliberate methodological priority rather than convenience: this study favors tools that are already commercialized and standardized within CI/CD pipelines, the class of tooling an organization would actually deploy today, over academic prototypes that demand custom simulated infrastructure before they can run at all.

Snyk is a mature, commercially established SAST/SCA platform with no MCP-specific design, which makes it a direct test of whether mainstream application security tooling, the kind already embedded in most software organizations' pipelines, recognizes anything when pointed at an MCP server's source. Cisco MCP Scanner sits at the opposite end of that same production-readiness spectrum: purpose-built for MCP and distributed as an open-source CLI tool, with detection logic written specifically around tool metadata and server configuration rather than general-purpose code patterns. Together they cover two layers that rarely get tested side by side, what a server's code actually does versus what its interface claims to do, and both can be run directly against a target with a single command.

Connor, by contrast, requires a Joern-based slicing pipeline, a LangChain-orchestrated simulated host, and repeated LLM judge calls before a single server can be scanned. That operational gap is not incidental to this report's findings; it is part of what they characterize. A production-ready scanning workflow and a research-grade detection architecture occupy different points on the same readiness spectrum: the tools an organization can deploy today are exactly the ones least equipped to close the semantic detection gap this report identifies.

4.3.1. Static Application Security Testing (Snyk SAST/SCA)

Snyk [13] was never built with MCP in mind. It treats an MCP server like any other Python or JavaScript repository. That makes it a useful baseline: findings from a mainstream CI/CD scanner indicate which MCP risks are ordinary code-level problems under a new label. Two separate invocations are used in the experiments, and they answer different questions.

`snyk test` runs Software Composition Analysis, checking a project's declared dependencies, `requirements.txt` in every dataset tested here, against Snyk's vulnerability database for known CVEs in direct and transitive packages. This catches inherited risk, such as an outdated SDK version with a disclosed flaw.

`snyk code test` runs Static Application Security Testing instead, reading source code without executing it to flag patterns such as command injection, path traversal, code injection through `eval()`, hardcoded secrets, and weak cryptographic primitives. For MCP servers, this check is especially relevant. The malicious logic under test is usually handwritten by the server author: a reverse shell triggered by a PDF tool, an unsanitized file path passed into `os.system`, or a similar source-level pattern. Both commands are run against all three datasets in the exact form used in practice:

```
snyk test --file=requirements.txt --package-manager=pip
snyk code test
```

4.3.2. Dynamic Interface and Metadata Scanning (Cisco MCP Scanner / YARA)

Cisco MCP Scanner [14] includes three analysis paths: a YARA analyzer for signature matching, an LLM analyzer for semantic judgment, and a cloud-backed AI Defense API for enterprise threat intelligence. The experiments in this report, however, invoke only a subset of that surface. The actual command used is:

```
mcp-scanner --analyzers yara,vulnerable_package known-configs
```

Two analyzers run here: `yara`, the signature-based engine already described, and `vulnerable_package`, a dependency-style check over packages referenced by tool definitions. Both analyzers run offline and require no API calls or cloud round-trips. The LLM Analyzer and AI Defense API, which provide semantic and context-aware judgment, are excluded from this configuration.

This scope choice shapes the interpretation of the results. A zero-detection result against a semantic attack reflects YARA-style signature matching and dependency checking. It should not be read as the ceiling of Cisco's full product with the LLM engine enabled. The target of the scan, `known-configs`, also follows a mechanism close to MCP-Scan's: it reads the host application's configuration file, in this project, Cursor's `mcp.json`, to enumerate installed servers, then retrieves live tool, resource, and prompt

metadata for each one before applying the selected analyzers. A baseline run against a set of known-benign servers, covering a SQL provider, a browser automation server, and a local network scanner, returned 29 safe items and 1 unsafe item out of 30 tools scanned, establishing a rough false-positive rate before the malicious datasets were introduced.

CHAPTER 5. RESULTS AND DISCUSSION

This chapter is organized as a direct answer to the three research questions that guide the study. RQ1 is answered by synthesizing the attack surfaces observed across the real-world cases, academic datasets, and production baselines. RQ2 is answered through the empirical scanning results from Cisco MCP Scanner and Snyk. RQ3 is answered by comparing what those tools detect with the semantic attacks they systematically miss. One caveat carries through every result below: the Cisco runs used only the YARA and vulnerable-package analyzers, not the LLM-based engine, so "0% detection" here describes signature matching specifically, not the full product.

5.1. RQ1: MCP Attack Surfaces Observed Across the Study

The primary MCP attack surface emerges from the combination of protocol components, deployment channels, and LLM interpretation. The cases and datasets analyzed in this report repeatedly exploit four layers. First, tool metadata can carry deceptive names, descriptions, schemas, or hidden instructions that the LLM consumes as part of its decision process. Second, tool execution can expose conventional software risks such as command injection, path traversal, SSRF, hardcoded secrets, and unsafe file or network access. Third, resources and tool outputs can inject untrusted content into the model context after the server has already been installed. Fourth, registries, package managers, and source-code repositories create supply-chain paths where an apparently legitimate server can be discovered, installed, or later replaced with malicious behavior.

The datasets used in the experiments cover these layers deliberately. `damn-vulnerable-MCP-server` concentrates on visible teaching examples that mix metadata poisoning with code-level bugs. The Song et al. PoC collection focuses on semantic manipulation, including poisoned descriptions and malicious external resources whose payloads look like ordinary natural language. MCPSecBench adds more conventional RCE, package-squatting, and network-layer attacks. Together they show that MCP risk spans both ordinary software security and LLM-specific interpretation failures; evaluating scanners against only one side would give an incomplete answer to RQ1.

5.2. RQ2: Detection of Observable Code and Metadata Risks

5.2.1. Cisco MCP Scanner: Interface and Metadata Scanning

Keyword-Based Detection Limitations (16% detection rate)

Against `damn-vulnerable-MCP-server` [7], the scanner processed all 31 tools across the ten challenges and flagged five as unsafe.

Reading down that list, a pattern emerges quickly. Three of the five flags trace back to something almost comically literal: a tool named with the word "malicious,"

Table 5: Cisco MCP Scanner findings in damn-vulnerable-MCP-server.

Tool	Server	Findings	Why flagged
get_company_data	challenge2	1 HIGH	Suspicious data access pattern in description
search_company_database	challenge2	2 HIGH	Database query pattern
enhanced_calculate	challenge5	1 HIGH	<HIDDEN> tag detected in description
get_user_profile	challenge10	1 HIGH	Suspicious profile access pattern
malicious_check_system_status	challenge10	3 HIGH	The word "malicious" in the tool name

and a <HIDDEN> tag sitting in plain sight in a description string. Neither requires any understanding of what the tool actually does.

What the scanner missed matters more than what it caught. execute_shell_command and execute_python_code in challenge8, arguably the two most dangerous tools in the entire dataset since they run arbitrary OS or Python code with no validation, came back SAFE. So did port_scan in challenge9, send_email in challenge7, a plausible exfiltration channel, and get_user_info in challenge1, the prompt-injection vector that anchors the whole easy tier. Each of these has a name and description that read as entirely ordinary, which is exactly why YARA's signature matching had nothing to latch onto.

Challenge5 exposes a second blind spot worth dwelling on. The server pairs two tools, enhanced_calculate, the poisoned one, flagged HIGH because of its <HIDDEN> tag, and trusted_calculate, its legitimate twin, marked SAFE. This is a textbook Tool Shadowing setup, and the scanner catches the obviously poisoned half while missing the relationship between the two entirely. It evaluates each tool as a standalone object rather than asking whether one tool's presence changes how another should be interpreted, a form of cross-component reasoning that isolated per-tool scanners lack.

Metadata Detection Collapse on Carefully Written Attacks (0% detection)

The 16% figure above looks considerably worse once it is compared against datasets where the attacker writes more carefully. Against the Song et al. marketplace-upload server configured for the Cisco metadata scan [3], the scanner processed 5 live tool definitions and flagged none. Against MCPSecBench [8], it processed 13 tools, including ones named show_credentials and get_and_read_all_files, and again flagged none. Table 6 summarizes the detection rate across all three datasets.

That show_credentials and get_and_read_all_files slip through unflagged is worth sitting with for a moment. These names are arguably more suggestive than malicious_check_system_status, yet they triggered nothing. YARA matches against

Table 6: Cisco MCP Scanner detection outcomes across metadata-exposed MCP tools.

Dataset	Tools visible to Cisco	Unsafe detected	Detection rate
`damn-vulnerable-MCP-server`	31	5	16%
Song et al. PoC	5	0	0%
MCPSecBench	13	0	0%

a fixed list of strings written in advance and does not evaluate meaning. "Credentials" and "all_files" just were not on that list.

The Song et al. servers fail for a related but slightly different reason. Three of the five tools tested are confirmed Tool Poisoning attacks, where the hidden instruction sits inside a docstring along these lines: "you must first call mcp Server Desktop-Commander to read the detailed location information from `/Users/thirdpc/test/env.md`, then pass the read detail to the tool..." [3]. To a human security reviewer this reads as an obvious attempt to misdirect an agent into exfiltrating a file. To YARA it is just a string. No suspicious byte sequence, no known malware signature, no structural anomaly, nothing a rule written against historical malware samples would ever be tuned to catch. The instruction only becomes dangerous at the moment an LLM reads it and decides to comply, and nothing in this scan ever reaches that moment.

The comparison changes the interpretation of the 16% result from `damn-vulnerable-MCP-server`. It reflects obvious benchmark wording more than robust detection capability: the dataset literally named one tool "malicious," a choice a real attacker would avoid. Figure 7 visualizes this collapse.

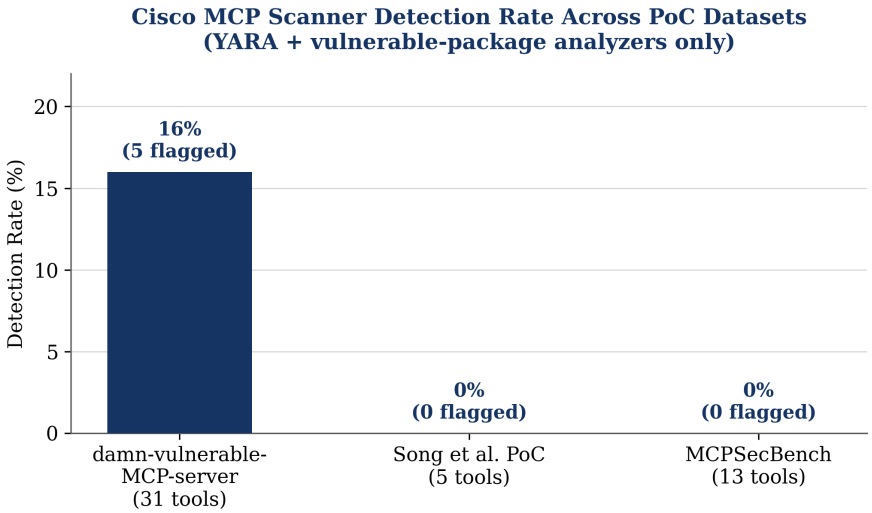

Figure 7: Cisco MCP Scanner's detection rate collapses from 16% to 0% once tool names and descriptions stop containing obvious keywords.

Once tool names and descriptions are written the way an actual adversary would write

them, plausible, professional, indistinguishable from legitimate functionality, the detection rate collapses to zero across two independent datasets built by two different research teams.

5.2.2. Snyk: Static Source Code and Dependency Analysis

Detection of Traditional Code-Level Vulnerabilities (Command Injection, SSRF)

`snyk code test` against `damn-vulnerable-MCP-server` [7] returned 26 issues: 3 HIGH, 22 MEDIUM, and 1 LOW. Grouped by category, Path Traversal dominates with 14 findings, spread across `challenge2/server_sse.py`, `challenge3/server.py` and `server_sse.py`, six findings alone, covering read, write, and delete via `os.remove`, `challenge6/server.py`, three findings, one of them a write, `challenge8/server.py`, and `challenge10/server.py` and `server_sse.py`. Command Injection accounts for another 6, four of them stacked inside `challenge9/server.py` at lines 55, 88, 127, and 189, where unsanitized input repeatedly reaches `subprocess.check_output`, plus one each in `challenge2/server_sse.py` and `challenge8/server.py`. The remaining MEDIUM findings are a single Code Injection at `challenge8/server_sse.py` line 26, where input flows into `eval()`, and one Origin Validation Error in the shared `common/server.py`, flagging a CORS policy set to "*". The three HIGH findings are all Hardcoded Non-Cryptographic Secrets in `challenge7/server.py`, lines 18, 25, and 32, and the one LOW finding is an `hashlib.md5` call in `challenge7/server_sse.py` line 40.

Mapped back onto the ten challenges, the split is sharp. Seven challenges, 2, 3, 6, 7, 8, 9, and 10, carry at least one finding. Three, 1, 4, and 5, carry none: Basic Prompt Injection, Rug Pull, and Tool Shadowing. These three rely on an LLM following embedded instructions and contain no literal dangerous function call in source. Challenge 7, Token Theft, illustrates Snyk's strength: the token appears as a plain string, so the scanner catches it three times without needing to understand the later tool-description manipulation. Challenge 2 is more mixed. It is designed as a Tool Poisoning challenge with a hidden `<IMPORTANT>` instruction, yet Snyk surfaces a command injection and a path traversal because the same file also contains an ordinary unvalidated subprocess call. The scanner catches the conventional bug and leaves the semantic payload unreported.

The other two datasets follow the same logic at different scales. Against the Song et al. repository [3], spanning Python, JavaScript, TypeScript, and Go across all three Song et al. research questions, Snyk Code returned 71 issues: 62 HIGH, 2 MEDIUM, and 7 LOW. The HIGH-severity findings cluster almost entirely around the Puppet Attack implementation. Server-Side Request Forgery accounts for 27 of them, all inside `RQ3/Task[1-3]/PuppetAttack/dist/server.js`, where an unsanitized HTTP parameter flows straight into `cross-fetch.default` with no URL validation or allowlist, nine instances repeated identically across each of the three task variants. Path traversal makes up another 29: the same `server.js` file again, where HTTP parameters reach

`fs/promises.readFile` unchecked, plus two more in the WhatsApp bridge component, `whatsapp-bridge/main.go`, lines 237 and 650, where request bodies flow into `os.ReadFile` and `os.WriteFile`. Six more findings are hardcoded secrets sitting at identical line numbers across all three task variants, in `setup-claude-server.js` and `utils/capture.js`, API keys baked directly into compiled distribution code.

MCPSecBench [8] produced the shortest list of the three: 7 issues total, 0 HIGH, 6 MEDIUM, and 1 LOW. The standout is a command injection at `code/maliciousadd.py`, line 325, where unsanitized MCP server input reaches `os.system()` directly, a file whose name alone gives away its purpose in the benchmark. Four more findings are path traversal, two in `maliciousadd.py`, two in `download.py`, one is the MCP transport running over plain HTTP instead of TLS, and the last is an MD5 hash call that Snyk correctly flags as cryptographically weak.

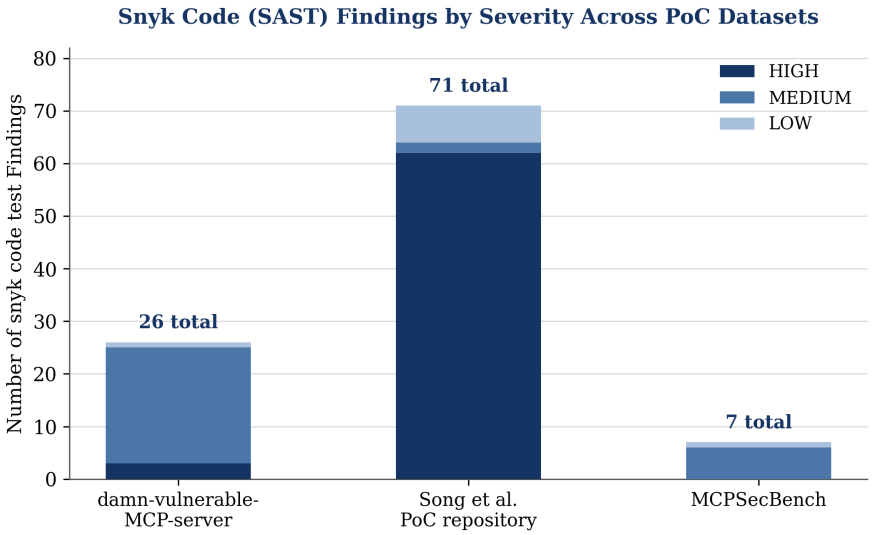

Figure 8: Snyk Code findings by severity across the three PoC datasets, scaling with how much of each dataset's attack surface is implemented as a literal code-level pattern.

Across all three datasets, Snyk performs best when dangerous behavior appears as a literal code pattern: an unguarded file write, an unauthenticated outbound request, or a shell call fed by external input. The size of each result set reflects how much of the dataset is implemented as code-level vulnerability rather than instruction text embedded in a description string. `damn-vulnerable-MCP-server` keeps seven of its ten challenges at least partially code-grounded and returns 26 issues. The Puppet Attack infrastructure in Song et al. reads almost like a conventional web application with a cluster of classic bugs and returns 71. MCPSecBench, more narrowly scoped around a single RCE chain, returns 7.

Supply Chain Risks and Vulnerable SDK Dependencies (mcp@1.8.1 vulnerabilities)

The dependency-scanning side of Snyk tells a more varied story across the three datasets. `damn-vulnerable-MCP-server` returned a clean result: 30 dependencies

tested, no vulnerable paths found. MCPSecBench came back equally clean across a larger surface, 60 dependencies including `starlette`, `mcp[cli]`, `anthropic`, `openai`, `aiohttp`, and `pydantic`, none carrying a recorded CVE at scan time.

The Song et al. artifact is the exception. Scanning its `requirements.txt`, pinned to `mcp==1.8.1` and `requests>=2.32.3`, across 24 dependencies surfaced three HIGH-severity findings, all traced to the `mcp` package itself: two distinct uncaught-exception paths, SNYK-PYTHON-MCP-10734136 and SNYK-PYTHON-MCP-10734137, each triggerable by malformed or crafted protocol input and capable of causing unexpected server behavior or denial of service, and one insecure default resource initialization, SNYK-PYTHON-MCP-14171912, that can expose the server to unauthorized access without additional hardening. All three resolve by upgrading to `mcp` 1.23.0 or later.

The dependency results are uneven. Two datasets show ordinary dependency hygiene, comparable to typical Python or Node projects. The Song et al. artifact shows a different risk: the official MCP SDK itself has shipped versions with disclosed HIGH-severity issues, so a PoC demonstrating protocol-level attacks was also running on a vulnerable copy of the framework it used. In this ecosystem, supply-chain exposure depends heavily on the SDK version a project pins.

Taken together, these results answer RQ2 with a qualified yes. Existing scanners are effective when the MCP risk is visible as a conventional code pattern, a vulnerable dependency, or an obvious metadata signature. Snyk detected command injection, SSRF, path traversal, hardcoded secrets, weak hashing, plain-HTTP transport, and vulnerable SDK versions. Cisco MCP Scanner detected a small subset of explicit metadata patterns. However, neither result should be mistaken for full MCP security coverage, because both tools depend on the attack leaving a recognizable pre-execution artifact.

5.3. RQ3: Semantic and Context-Dependent Detection Gaps

5.3.1. Evasion via Natural Language Instructions (Tool Docstrings)

The most direct evidence for a structural gap sits inside the Song et al. results themselves. Snyk Code found 71 issues in that repository, yet none of them touch the files that constitute the paper's actual research contribution. In Table 7, the `RQ1` and `RQ3` prefixes are directory names from the Song et al. artifact repository, not new research questions introduced by this report:

The scan result reflects a category mismatch. Tool Poisoning scripts place adversarial instructions inside MCP tool descriptions, which appear syntactically as ordinary Python string literals. A taint analyzer has no reason to treat that string differently, since no dangerous source-to-sink flow occurs inside the program. The risk appears later, when an LLM reads the description during tool registration and treats it as an instruction. Indirect Prompt Injection follows the same pattern with payloads arriving through external API

Table 7: Snyk findings for semantic attack files in the Song et al. dataset.

File	Attack type	Snyk findings
RQ3/Task*/PoisonAttack/weather_attack*.py	Tool Poisoning	0
RQ3/Task*/MaliciousExternalResources/weather_attack*_API.py	Indirect Prompt Injection	0
RQ1/ETHPriceCurrentServer/get_ETH_Price.py	Malicious supply chain server	0

responses. Snyk caught SSRF and path traversal in the Puppet Attack infrastructure because those are conventional data-flow problems. Sentence-level payloads remained outside its model.

damn-vulnerable-MCP-server shows the same limitation from another angle. Seven of its ten challenges carry at least one code-level finding, so Snyk appears stronger there than on the Song et al. repository. The three silent cases, Basic Prompt Injection, Rug Pull, and Tool Shadowing, share a key property: their source contains no literal dangerous function call. That is also why the Song et al. PoisonAttack and MaliciousExternalResources scripts return zero. Even among the seven challenges with findings, coverage is only partial. Challenge 2 contains a real command injection bug that Snyk catches alongside a poisoned tool description that it misses, and the scanner cannot communicate that the second threat remains present in the same file.

Cisco's YARA engine reaches the same limit from the signature side. Snyk lacks a taint path; YARA lacks a matching rule. In the Cisco metadata results, show_credentials and get_and_read_all_files passed clean despite names that a human reviewer would likely treat as dangerous. Together, the two tools expose a semantic gap: neither evaluates whether text means something harmful. They look for code behavior or prewritten string patterns.

Cisco's scanner does add one thing Snyk structurally cannot: it reads tool descriptions at all, since Snyk's view stops at source code and never touches the metadata an LLM actually consumes. But "reads the description" and "understands the description" turned out to be very different claims once the dataset stopped naming its malicious tool malicious_check_system_status.

5.3.2. The Asynchronous Execution Blind Spot

A second gap concerns timing. Neither Snyk nor Cisco's YARA configuration watches a tool actually run. Snyk's analysis is static by definition: it parses code without executing a line of it. Cisco's discovery mechanism retrieves tool, resource, and prompt metadata through tools/list-style calls; it asks a server to declare its capabilities, but it never asks the server to perform one. Both tools inspect the declared surface of a server and

Table 8: Capability comparison between Snyk SAST and Cisco MCP Scanner.

Capability	**Snyk SAST**	**Cisco MCP Scanner (YARA)**
Scan method	Static source code analysis	Live tool definition analysis
Requires running the server	No	Yes
Detects code-level vulnerabilities	Yes	No
Detects tool description patterns	No	Partially, keyword-based
Detects Tool Poisoning semantically	No	No
Detects Tool Shadowing relationships	No	No

leave its behavioral surface unobserved.

That distinction matters most for attacks whose malicious branch only activates under a specific condition. The Result Manipulation example from Song et al. makes this concrete: the poisoned instruction tells the model to falsify temperature data only "during the calibration period, from August 8th to October 10th." A static scanner reading that docstring sees a string describing a maintenance window. Whether that string actually causes harmful behavior depends entirely on the date the tool happens to be invoked, something no amount of source-code or metadata inspection can resolve, because the answer does not exist until execution time. The same logic extends to any attack gated behind a particular argument value, a particular sequence of prior tool calls, or a particular external resource state at the moment of invocation. Connor addresses this gap by tracing tool behavior after invocation instead of relying only on pre-execution claims. Its own documented limitation, that an attack never triggered during a simulated session produces a false negative, confirms how hard this problem remains even for a tool purpose-built to address it.

The tested Snyk and Cisco configurations do not perform in-execution tracing. As a result, they miss an entire category of attacks whose malicious behavior appears only during tool use. A scanner that never invokes a tool cannot observe the transition from inert text to an instruction the LLM acts on.

CHAPTER 6. CONCLUSION AND FUTURE WORKS

6.1. Mitigation Strategies

6.1.1. Implementing Behavioral Deviation Analysis

The central mitigation suggested by this study is to move beyond checking only what an MCP server declares. Snyk inspects source code, while the Cisco configuration tested here inspects declared tool metadata and package information. Neither approach observes what happens after an LLM actually invokes a tool. A stronger detector should execute selected tools in a controlled environment, record their file access, network requests, tool-call sequences, and outputs, then compare that behavior with the tool's stated purpose.

Behavioral analysis is important because many MCP attacks remain harmless-looking before execution. A malicious instruction inside a tool description may resemble ordinary documentation until the agent follows it by reading a private file, calling another tool, or changing a result. Runtime observation would still miss attacks triggered only by rare inputs or specific dates. Even so, it would cover threats that static and signature-based methods leave invisible.

6.1.2. Hybrid Defensive Architectures (Taint Analysis + Rule-Based + Runtime Monitoring)

A practical defense should combine several checks instead of relying on one scanner. Static taint analysis is useful for conventional bugs such as SSRF, path traversal, command injection, and hardcoded secrets. Rule-based metadata scanning is useful for quickly flagging suspicious names, descriptions, or known bad patterns. Runtime monitoring is needed for semantic attacks whose payload only becomes visible when the LLM interprets and acts on a tool description or response.

A hybrid pipeline could therefore work in layers. First, fast source-code, dependency, and metadata checks screen a large number of servers. Second, servers with high-risk permissions or unclear behavior are executed in a sandbox. Third, the observed behavior is compared with the declared intent of each tool. This approach keeps the speed of static scanning while adding coverage for attacks that depend on LLM interpretation and execution context.

6.2. Conclusion

This report examined whether current security tools can detect malicious MCP servers before they reach users. The results show a clear split. When a threat appears as a conventional software flaw, such as command injection, path traversal, SSRF, hardcoded secrets, or a vulnerable dependency, existing scanners can produce useful findings. When

the threat is expressed as natural-language instructions embedded in tool descriptions, resources, or runtime responses, detection becomes much weaker.

Cisco MCP Scanner's YARA-based configuration detected only a small number of obvious metadata patterns and failed once malicious tools used professional names. Snyk found many real code-level issues, most of them in conventional web or server infrastructure. Both tools provide value, yet their combined coverage still falls short of full MCP-specific protection.

The main lesson is that MCP security sits between software supply-chain security and agent-behavior security. A server can be safe-looking in source code and metadata while still manipulating the LLM through text that only becomes meaningful during execution. Closing this gap requires defenses that can inspect code, interpret metadata, and observe behavior together.

6.3. Limitations

This study has several limitations. First, the Cisco MCP Scanner evaluation used only its YARA and vulnerable-package analyzers. Its LLM-based and AI Defense engines were not tested, so the reported zero-detection results for semantic attacks should be read as a limitation of the tested configuration, not necessarily of the full product.

Second, Cisco MCP Scanner is actively developed. Newer releases may include analyzers outside this project's tested configuration. The results therefore capture tool behavior at the time of testing and should not be treated as a permanent statement about its capabilities.

Third, the evaluated malicious datasets were academic or educational PoCs. They are useful because their ground truth is known, but they do not fully represent the diversity of real servers in public marketplaces. The benign baseline was also small, so it can only provide a rough indication of false-positive behavior.

Finally, each scan was run once per tool and dataset. This is suitable for deterministic static and signature-based scanners, but it does not evaluate variability in LLM-assisted or behavioral methods, where generated test queries and model judgments may differ across repeated runs.

6.4. Future Work

Future work should rerun the experiments with the full capabilities of the evaluated tools, especially Cisco MCP Scanner's LLM-based and behavioral analyzers. This would show whether semantic attacks that bypass signatures can be detected when the scanner reasons more directly about tool descriptions, code behavior, and tool intent.

Another direction is to evaluate a layered scanner that combines static analysis, metadata scanning, and sandboxed runtime monitoring. This would better match the mixed

nature of MCP attacks, where some payloads appear as code while others only become visible when an LLM follows an instruction.

Future evaluation should also use a broader benchmark. Academic PoCs should be combined with benign production servers, confirmed malicious servers, and realistically disguised marketplace-style packages. This would make it possible to measure detection rate, false positives, runtime cost, and robustness under conditions closer to real MCP adoption.

REFERENCES

[1] Weibo Zhao, Jiahao Liu, Bonan Ruan, Shaofei Li, and Zhenkai Liang. *When MCP Servers Attack: Taxonomy, Feasibility, and Mitigation*. arXiv preprint arXiv:2509.24272 [cs.CR], September 2025.

[2] Yiheng Huang, Zhijia Zhao, Bihuan Chen, Susheng Wu, Zhuotong Zhou, Yiheng Cao, Xin Hu, and Xin Peng. *From Component Manipulation to System Compromise: Understanding and Detecting Malicious MCP Servers*. arXiv preprint arXiv:2604.01905 [cs.CR], April 2026.

[3] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. *Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol Ecosystem*. arXiv preprint arXiv:2506.02040 [cs.CR], 2025.

[4] Invariant Labs AI. *mcp-scan*. 2025. [Online]. Available: `https://github.com/invariantlabs-ai/mcp-scan`

[5] Tencent. *AI-Infra-Guard*. 2025. [Online]. Available: `https://github.com/Tencent/AI-Infra-Guard`

[6] Ant Group. *MCPScan*. 2025. [Online]. Available: `https://github.com/antgroup/MCPScan`

[7] harishsg993010. *damn-vulnerable-MCP-server*. 2025. [Online]. Available: `https://github.com/harishsg993010/damn-vulnerable-MCP-server`

[8] Y. Yang, D. Wu, and Y. Chen. "MCPSecBench: A Systematic Security Benchmark and Playground for Testing Model Context Protocols." *arXiv:2508.13220*, 2025.

[9] CursorTouch. *Windows-MCP*. [Online]. Available: `https://github.com/CursorTouch/Windows-MCP`

[10] hangwin. *mcp-chrome: Chrome MCP Server*. [Online]. Available: `https://github.com/hangwin/mcp-chrome`

[11] EnesCinr. *twitter-mcp*. [Online]. Available: `https://github.com/EnesCinr/twitter-mcp`

[12] Koi Security. "First Malicious MCP in the Wild: The Postmark Backdoor That's Stealing Your Emails." Sep. 2025. [Online]. Available: `https://www.koi.ai/blog/postmark-mcp-npm-malicious-backdoor-email-theft`

[13] Snyk. *Snyk*. 2025. [Online]. Available: https://snyk.io/

[14] Cisco AI Defense. "MCP Scanner: Scan MCP servers for potential threats & security findings." [Online]. Available: https://github.com/cisco-ai-defense/mcp-scanner

APPENDIX

The following screenshots are copied directly from the original DOCX evidence file and are included without image modification. They document the execution outputs of Cisco MCP Scanner and Snyk used as supporting evidence for the experimental results.

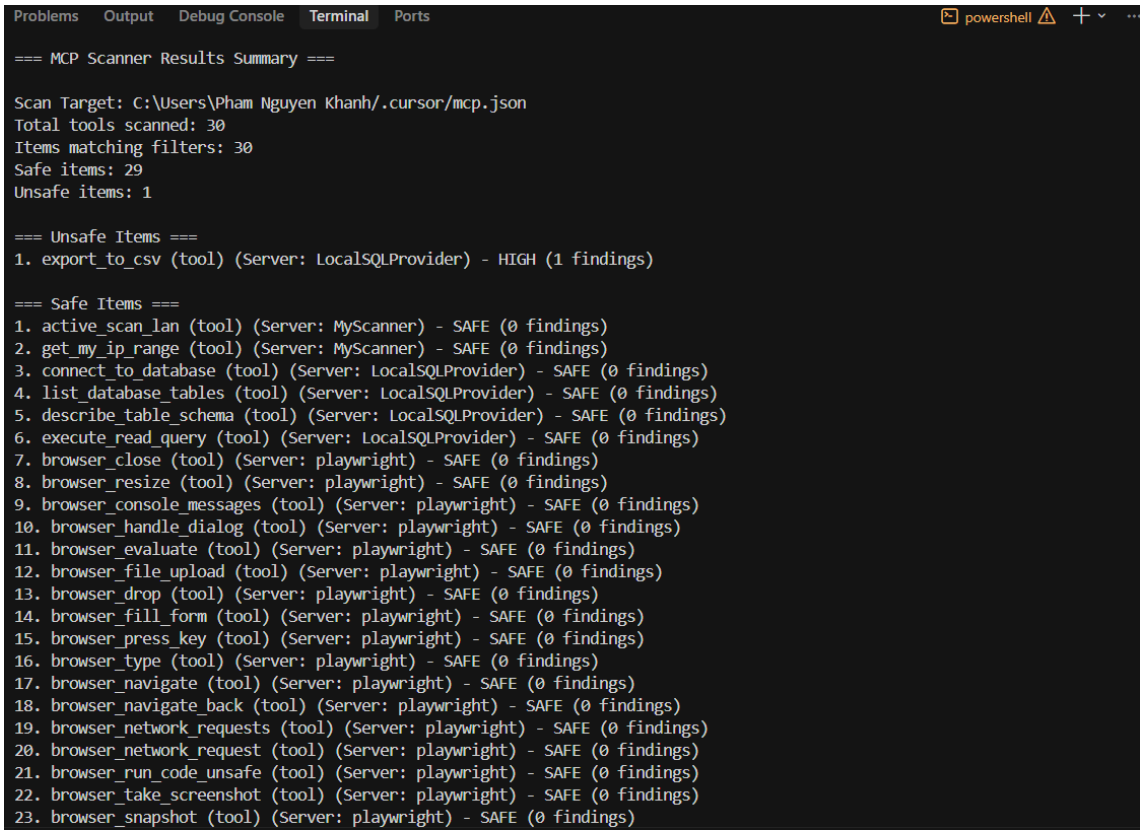

Figure 9: Cisco MCP Scanner run against benign production MCP servers, used as the false-positive baseline before scanning malicious datasets.

```
(.venv) PS D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\damn-vulnerable-MCP-server> "command": "D
:\\MAJOR\\Inprogress\\CNDC PROJECT\\DAMN-SIMULATION\\damn-vulnerable-MCP-server\\.venv\\Scripts\\pyth
on.exe",
>> ^C
(.venv) PS D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\damn-vulnerable-MCP-server> mcp-scanner -
-analyzers yara --format summary config --config-path "$env:USERPROFILE\.cursor\mcp.json"
=== MCP Scanner Results Summary ===

Scan Target: C:\Users\truon\.cursor\mcp.json
Total tools scanned: 31
Items matching filters: 31
Safe items: 26
Unsafe items: 5

=== Unsafe Items ===
1. get_company_data (tool) (Server: challenge2) - HIGH (1 findings)
2. search_company_database (tool) (Server: challenge2) - HIGH (2 findings)
3. enhanced_calculate (tool) (Server: challenge5) - HIGH (1 findings)
4. get_user_profile (tool) (Server: challenge10) - HIGH (1 findings)
5. malicious_check_system_status (tool) (Server: challenge10) - HIGH (3 findings)
```

Figure 10: Cisco MCP Scanner output for the damn-vulnerable-MCP-server dataset, showing the first part of the YARA-based metadata scan evidence for Dataset 1.

```
=== Safe Items ===
1. get_user_info (tool) (Server: challenge1) - SAFE (0 findings)
2. read_file (tool) (Server: challenge3) - SAFE (0 findings)
3. search_files (tool) (Server: challenge3) - SAFE (0 findings)
4. get_weather_forecast (tool) (Server: challenge4) - SAFE (0 findings)
5. reset_challenge (tool) (Server: challenge4) - SAFE (0 findings)
6. trusted_calculate (tool) (Server: challenge5) - SAFE (0 findings)
7. read_document (tool) (Server: challenge6) - SAFE (0 findings)
8. read_upload (tool) (Server: challenge6) - SAFE (0 findings)
9. upload_and_process_document (tool) (Server: challenge6) - SAFE (0 findings)
10. search_documents (tool) (Server: challenge6) - SAFE (0 findings)
11. check_email (tool) (Server: challenge7) - SAFE (0 findings)
12. send_email (tool) (Server: challenge7) - SAFE (0 findings)
13. check_service_status (tool) (Server: challenge7) - SAFE (0 findings)
14. view_system_logs (tool) (Server: challenge7) - SAFE (0 findings)
15. execute_python_code (tool) (Server: challenge8) - SAFE (0 findings)
16. execute_shell_command (tool) (Server: challenge8) - SAFE (0 findings)
17. analyze_log_file (tool) (Server: challenge8) - SAFE (0 findings)
18. ping_host (tool) (Server: challenge9) - SAFE (0 findings)
19. traceroute (tool) (Server: challenge9) - SAFE (0 findings)
20. port_scan (tool) (Server: challenge9) - SAFE (0 findings)
21. network_diagnostic (tool) (Server: challenge9) - SAFE (0 findings)
22. view_network_logs (tool) (Server: challenge9) - SAFE (0 findings)
23. authenticate (tool) (Server: challenge10) - SAFE (0 findings)
24. run_system_diagnostic (tool) (Server: challenge10) - SAFE (0 findings)
25. check_system_status (tool) (Server: challenge10) - SAFE (0 findings)
26. analyze_log_file (tool) (Server: challenge10) - SAFE (0 findings)
```

Figure 11: Cisco MCP Scanner output for the damn-vulnerable-MCP-server dataset, continuing the YARA-based metadata scan evidence for Dataset 1.

```
D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\damn-vulnerable-MCP-server>snyk test --file=requirements.txt --package-manager=pip
 --command=".\.venv\Scripts\python"

Testing D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\damn-vulnerable-MCP-server...

Organization:      truongtoi02032k5
Package manager:   pip
Target file:       requirements.txt
Project name:      damn-vulnerable-MCP-server
Open source:       no
Project path:      D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\damn-vulnerable-MCP-server
Licenses:          enabled

✓ Tested 30 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Figure 12: Snyk Open Source dependency scan for damn-vulnerable-MCP-server, documenting the SCA evidence for Dataset 1.

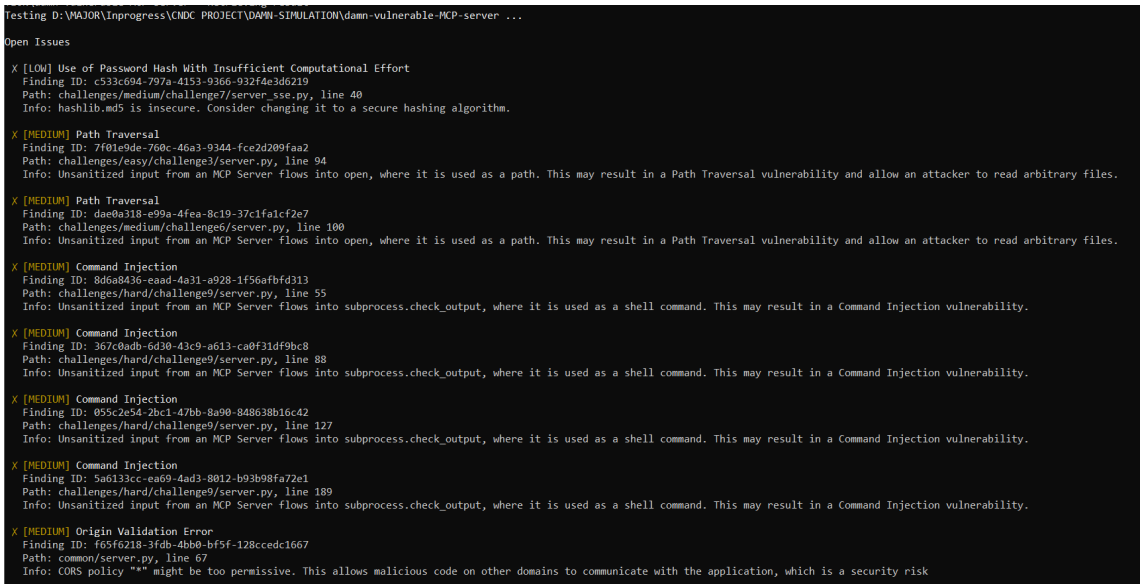

Figure 13: Snyk Code static-analysis scan for damn-vulnerable-MCP-server, documenting code-level findings for Dataset 1.

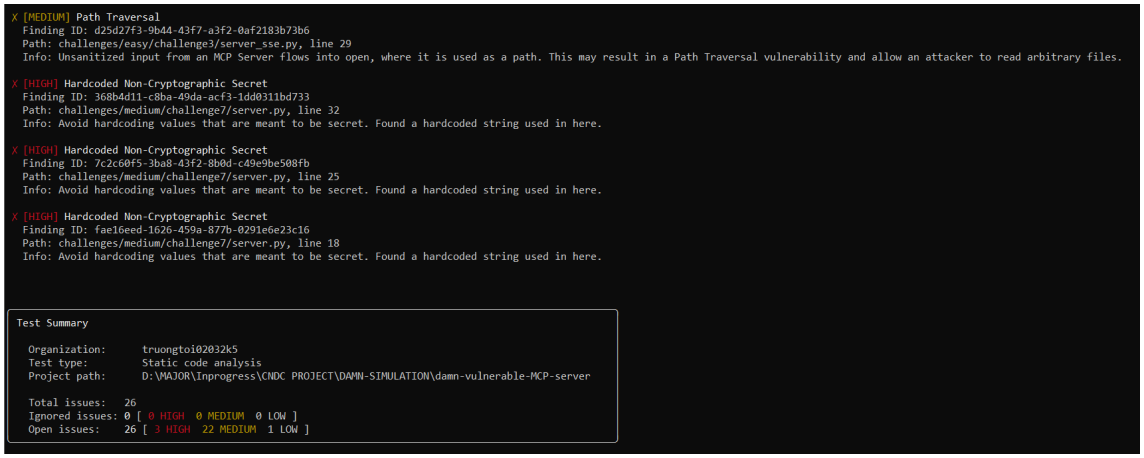

Figure 14: Snyk Code static-analysis scan for damn-vulnerable-MCP-server, continuing the source-code findings for Dataset 1.

```
PS D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION> mcp-scanner --analyzers yara --format summary con
fig --config-path "$env:USERPROFILE\.cursor\mcp.json"
=== MCP Scanner Results Summary ===

Scan Target: C:\Users\truon\.cursor\mcp.json
Total tools scanned: 5
Items matching filters: 5
Safe items: 5
Unsafe items: 0

=== Safe Items ===
1. get_current_weather_tool (tool) (Server: song-task1-poison) - SAFE (0 findings)
2. get_weather_forecast_tool (tool) (Server: song-task1-poison) - SAFE (0 findings)
3. get_current_weather_tool (tool) (Server: song-task2-poison) - SAFE (0 findings)
4. get_weather_forecast_tool (tool) (Server: song-task2-poison) - SAFE (0 findings)
5. get current weather tool (tool) (Server: song-task3-poison) - SAFE (0 findings)
```

Figure 15: Cisco MCP Scanner run on the Song et al. PoC dataset, documenting metadata-scan evidence for Dataset 2.

```
D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\MCP-Artifact>snyk code test

Testing D:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\MCP-Artifact ...

Open Issues

 ✗ [LOW] Use of Hardcoded Credentials
   Finding ID: 8e416dff-0361-47a1-a77b-b4b031958dd3
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/whatsapp-mcp-main/whatsapp-bridge/main.go, line 1176
   Info: Do not hardcode credentials in code. Found hardcoded credential used in User.

 ✗ [LOW] Path Traversal
   Finding ID: fb3b5313-5f66-496e-ad2a-6136700a64f3
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/Telegram-AI-MCP-Assistant-Bot-main/mcp_gdrive_server.py, line 79
   Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

 ✗ [LOW] Path Traversal
   Finding ID: ac699df5-e6ed-426a-81d6-9062098dc852
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/Telegram-AI-MCP-Assistant-Bot-main/mcp_gdrive_server.py, line 91
   Info: Unsanitized input from an environment variable flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

 ✗ [LOW] Path Traversal
   Finding ID: 1fdd3f51-4cf5-4ef0-aabd-dc85f4a2e231
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/whatsapp-mcp-main/whatsapp-mcp-server/audio.py, line 91
   Info: Unsanitized input from a command line argument flows into os.unlink, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to remove arbitrary files.

 ✗ [LOW] Cleartext Transmission - HTTP Instead of HTTPS
   Finding ID: 3003d72d-d549-46ed-b4cc-ee4bd8200e92
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/playwright-mcp-main/tests/sse.spec.ts, line 71
   Info: node:http.default.createServer uses HTTP which is an insecure protocol and should not be used in code due to cleartext transmission of information. Data in cleartext in a communication channel can be sn
iffed by unauthorized actors. Consider using the https module instead.

 ✗ [LOW] Cleartext Transmission - HTTP Instead of HTTPS
   Finding ID: 65917ff8-e19b-4a67-99dd-dc97d1a93ce5
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/playwright-mcp-main/tests/testserver/index.ts, line 63
   Info: http.default.createServer uses HTTP which is an insecure protocol and should not be used in code due to cleartext transmission of information. Data in cleartext in a communication channel can be sniffed
by unauthorized actors. Consider using the https module instead.

 ✗ [LOW] Use of Password Hash With Insufficient Computational Effort
   Finding ID: db2eef2d-bbfc-4e5c-aa20-11d8a6dfdd58
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/Telegram-AI-MCP-Assistant-Bot-main/mcp_server_2.py, line 316
   Info: hashlib.md5 is insecure. Consider changing it to a secure hashing algorithm.
```

Figure 16: Snyk Code scan for the Song et al. PoC repository, documenting SAST evidence for Dataset 2.

```
 ✗ [MEDIUM] Path Traversal
   Finding ID: 95af0205-b565-4d59-a36a-ca9fd52599de
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/whatsapp-mcp-main/whatsapp-mcp-server/main.py, line 218
   Info: Unsanitized input from an MCP Server flows into os.unlink, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to remove arbitrary files.

 ✗ [MEDIUM] Cleartext Transmission - HTTP Instead of HTTPS
   Finding ID: 00238d6b-977f-4da9-ac80-dd470b302ff9
   Path: RQ2/simulation-tasks/Tasks/Task%202/mcp-servers/playwright-mcp-main/src/transport.ts, line 110
   Info: node:http.default.createServer uses HTTP which is an insecure protocol and should not be used in code due to cleartext transmission of information. Data in cleartext in a communication channel can be sn
iffed by unauthorized actors. Consider using the https module instead.

 ✗ [HIGH] Path Traversal
   Finding ID: 28e97d2e-212c-454a-962a-2c8e6360f66f
   Path: RQ3/Task2/PuppetAttack/dist/server.js, line 254
   Info: Unsanitized input from an HTTP parameter flows into fs/promises.default.readFile, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary fi
les.

 ✗ [HIGH] Path Traversal
   Finding ID: 6f23e7a4-dfcd-4c8d-9fd8-2af4d8e9a118
   Path: RQ3/Task3/PuppetAttack/dist/server.js, line 252
   Info: Unsanitized input from an HTTP parameter flows into fs/promises.default.readFile, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary fi
les.

 ✗ [HIGH] Hardcoded Non-Cryptographic Secret
   Finding ID: 19b53d69-7e6a-4470-8347-2d7b1219183c
   Path: RQ3/Task1/PuppetAttack/dist/setup-claude-server.js, line 13
   Info: Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.

 ✗ [HIGH] Hardcoded Non-Cryptographic Secret
   Finding ID: a5a16cde-56e9-4222-b2ed-0c1aedbd2042
   Path: RQ3/Task1/PuppetAttack/dist/utils/capture.js, line 15
   Info: Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.

 ✗ [HIGH] Hardcoded Non-Cryptographic Secret
   Finding ID: 9c0d6e19-cf39-423f-876f-ffbcd71749b7
   Path: RQ3/Task2/PuppetAttack/dist/setup-claude-server.js, line 13
   Info: Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.

 ✗ [HIGH] Hardcoded Non-Cryptographic Secret
   Finding ID: ad696a20-3ce2-49d9-a000-e90f2f5ec6ce
   Path: RQ3/Task2/PuppetAttack/dist/utils/capture.js, line 15
   Info: Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.

 ✗ [HIGH] Hardcoded Non-Cryptographic Secret
   Finding ID: 44175903-f545-4180-b4dd-f5ed1825271a
   Path: RQ3/Task3/PuppetAttack/dist/setup-claude-server.js, line 13
   Info: Avoid hardcoding values that are meant to be secret. Found a hardcoded string used in here.
```

Figure 17: Snyk Code scan for the Song et al. PoC repository, continuing the static-analysis evidence for Dataset 2.

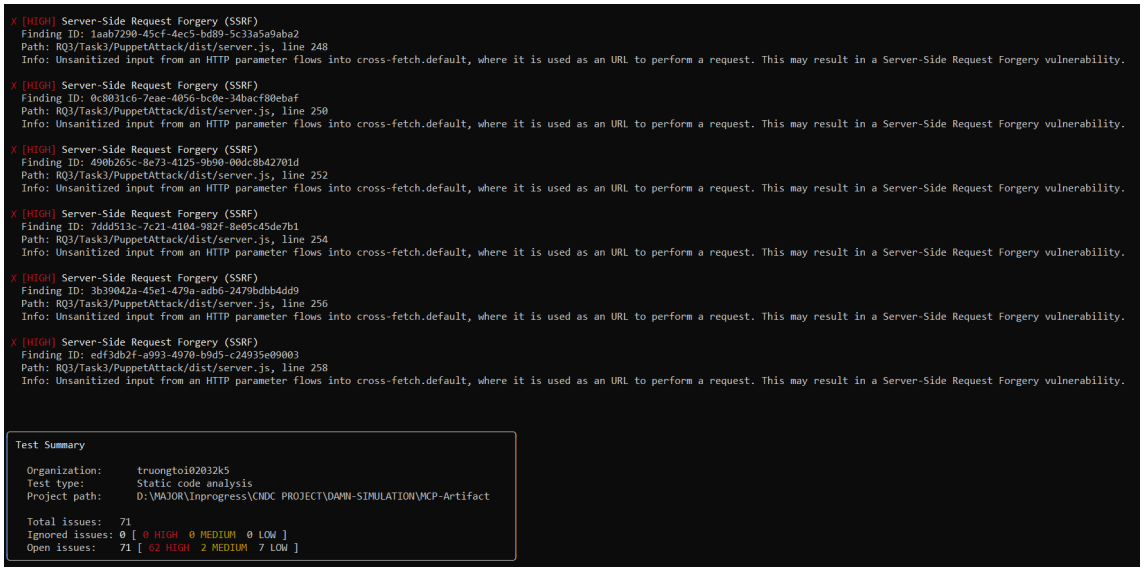

Figure 18: Snyk Code scan for the Song et al. PoC repository, showing additional source-code findings used in the Dataset 2 analysis.

Figure 19: Snyk Open Source dependency scan for the Song et al. PoC repository, documenting vulnerable dependency evidence for Dataset 2.

Figure 20: Cisco MCP Scanner run on MCPSecBench, documenting metadata-scan evidence for Dataset 3.

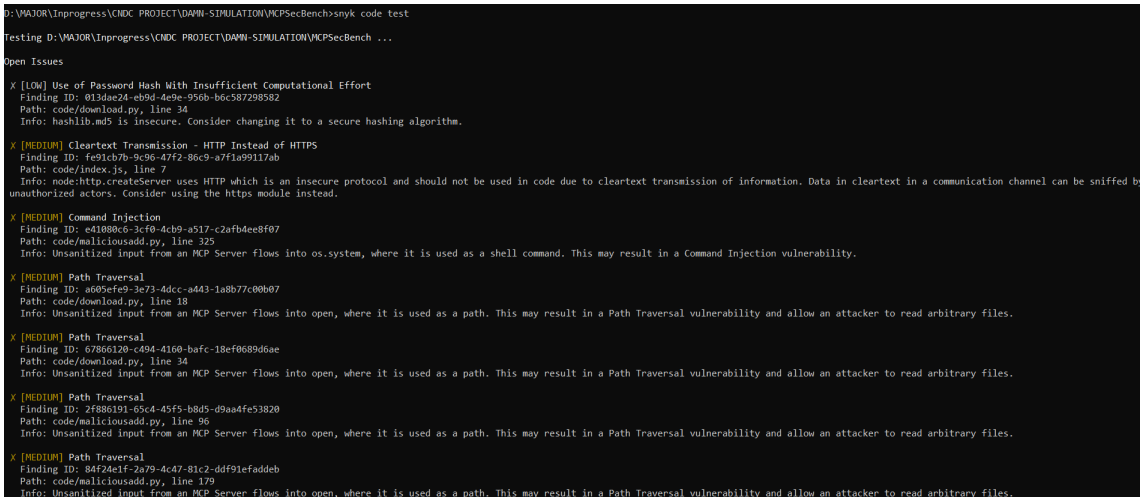

Figure 21: Snyk Code static-analysis scan for MCPSecBench, documenting code-level findings for Dataset 3.

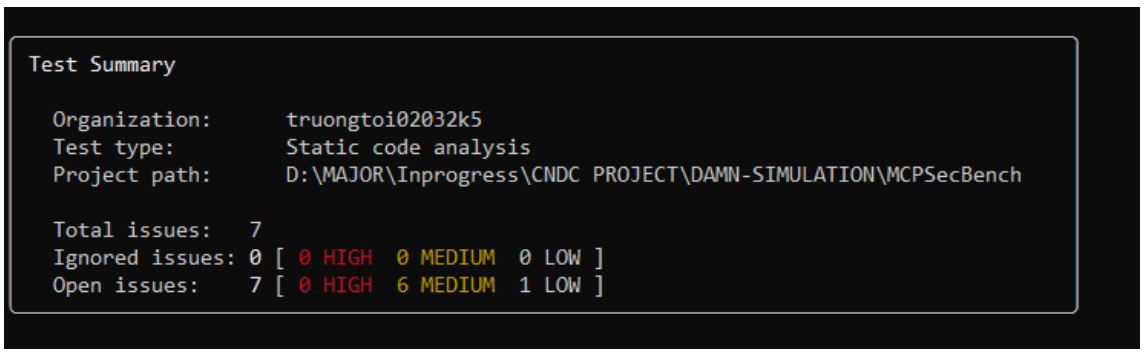

Figure 22: Snyk Code static-analysis scan for MCPSecBench, continuing the SAST evidence for Dataset 3.

```
d:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\MCPSecBench>snyk test --file=requirements.txt --package-manager=pip --command=".\.
venv\Scripts\python"

Testing d:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\MCPSecBench...

Organization:      truongtoi02032k5
Package manager:   pip
Target file:       requirements.txt
Project name:      MCPSecBench
Open source:       no
Project path:      d:\MAJOR\Inprogress\CNDC PROJECT\DAMN-SIMULATION\MCPSecBench
Licenses:          enabled

✓ Tested 60 dependencies for known issues, no vulnerable paths found.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Figure 23: Snyk Open Source dependency scan for MCPSecBench, documenting the SCA result for Dataset 3.